

# ספר פרויקט גמר - Simple Shop

## עמוד שער

**שם המכללה:** [שם המכללה] **סמליל (לוגו):** [לוגו המכללה] | [לוגו מה"ט] **שם מגמה:** הנדסת תוכנה **מסלול**  
**הכשרה:** הנדסאי **נושא הפרויקט:**

# מערכת E-Commerce מלאה לניהול חנות אונליין

עם תשלומים מאובטחים, ניהול מלאי בזמן אמת, ולוח ניהול מתקדם

שמות הסטודנטים: - [שם סטודנט 1] - צד שרת (Backend API) - [שם סטודנט 2] - צד לקוח (Frontend Application) שם המנחה האישי: [שם המנחה] תאריך מסירה: 10/05/2026 ---

## תוכן עניינים

### חלק א' - מבוא והצגת הפרויקט

1. תיאור ורקע כללי 2. מטרות המערכת 3. סקירת מצב קיים בשוק 4. מה הפרויקט מחדש ומשפר

### חלק ב' - ליבה טכנית (Server)

5. דרישות מערכת ופונקציונאליות 6. בעיות צפויות ופתרונות 7. פתרון טכנולוגי נבחר 8. מבני נתונים וארגון קבצים 9. תרשימי מערכת 10. מרכיב אלגוריתמי 11. אבטחת מידע

### חלק ג' - ליבה טכנית (Client)

12. צד לקוח - סקירה

### חלק ד' - ניהול פרויקט

13. משאבים נדרשים 14. תכנית עבודה 15. תכנון בדיקות 16. בקרת גרסאות 17. תכנית הפקה סופית לספר 18. קטעי קוד מומלצים להטמעה

### חלק ה' - סיכום

19. מה למדנו ---

# חלק א' - מבוא והצגת הפרויקט

## 1.3.1 תיאור ורקע כללי

### רקע והקשר

עם התרחבות תחום המסחר האלקטרוני, עסקים קטנים ובינוניים נדרשים להציע חוויית רכישה מקוונת אמינה, מאובטחת ונוחה לשימוש. במקרים רבים, הפתרונות הקיימים בשוק הם יקרים, מורכבים לתחזוקה, או מגבילים מבחינת התאמה לצרכים מקומיים וספציפיים של העסק. פרויקט **Simple Shop** פותח כמענה לצורך זה, מתוך מטרה לבנות מערכת מסחר אלקטרוני מלאה המדגימה תכנון נכון, אבטחת מידע, חלוקת אחריות בין צד שרת וצד לקוח, ושילוב שירותים חיצוניים בסביבת ייצור.

### תיאור המערכת

Simple Shop היא מערכת מסחר אלקטרוני מלאה הכוללת רכיבי ליבה הנדרשים להפעלת חנות מקוונת: - קטלוג מוצרים עם שליפה, סינון וחיפוש - עגלת קניות וניהול הזמנות - אינטגרציית תשלומים באמצעות Stripe - ניהול משתמשים, אימות והרשאות - ממשק ניהולי למוצרים, הזמנות וסטטיסטיקות - שכבות אבטחה להגנה על תהליכי התחברות, תשלום וצריכת API

### חשיבות הפרויקט

< "בשנת 2025, 21% מכלל הרכישות בעולם בוצעו אונליין. עד 2027, המספר צפוי לעלות ל-25%." > — Statista, E-commerce Worldwide Report מכאן עולה הצורך במערכות מסחר אלקטרוני שאינן רק פונקציונאליות, אלא גם מאובטחות, מתועדות וניתנות להרחבה. במסגרת פרויקט זה בחרנו לממש מערכת כזו באופן המדגים תהליך פיתוח מלא משלב התכנון ועד לשלב התייעוד והבדיקות. ---

## 1.3.2 מטרות המערכת

### מטרת העל

מטרת העל של הפרויקט היא לפתח מערכת חנות מקוונת עבור עסק קטן או בינוני, תוך שמירה על שלושה עקרונות מרכזיים: - פשטות תפעולית למשתמש הקצה ולמנהל המערכת - אבטחה מספקת עבור תהליכי אימות, רכישה ותשלום - גמישות ארכיטקטונית המאפשרת תחזוקה והרחבה עתידית

### מטרות מדידות

**זמינות** | 99.9% uptime שנתי | | **אבטחה** | 0 פרצות | penetration testing | | **קלות שימוש** | > 5 דקות | זמן  
ללמוד את המערכת | | **כיסוי בדיקות** | < 85% | test coverage |

## מטרות למידה

מעבר למימוש הפונקציונלי של המערכת, הוגדרו גם מטרות למידה מקצועיות: - תכנון ארכיטקטורה מודולרית וברת תחזוקה - יישום מנגנוני אבטחה במספר שכבות - עבודה עם שירותים חיצוניים כגון Stripe, Redis ו-MongoDB Atlas - כתיבת בדיקות אוטומטיות ותמיכה בתהליכי אימות איכות - הכנת תיעוד טכני מסודר עבור המערכת ---

## 1.3.3 סקירת מצב קיים בשוק

### מצב קיים

עסקים קטנים המעוניינים למכור באינטרנט ניצבים בפני מספר חלופות עיקריות: אפשרות | יתרון | חיסרון |  
|-----|-----|-----| | **Shopify / Wix** | מוכן מהקופסה | עלות חודשית גבוהה (100-500\$) + עמלות |  
**WooCommerce** | חינומי, גמיש | דורש ידע טכני, איטי, פגיע לפריצות | | **Magento** | עוצמתי, מלא תכונות |  
מורכב מאוד, דורש שרת חזק | | **בנייה מאפס** | שליטה מלאה | זמן פיתוח עצום, סיכון לשגיאות |

### פער מזוהה

קיים פער בין פלטפורמות מסחר מוכנות אך יקרות ומוגבלות, לבין פיתוח עצמאי מלא המצריך השקעת זמן ומשאבים רבים. הפרויקט נועד להדגים פתרון ביניים: מערכת מודולרית, מבוססת קוד, הניתנת ללמידה, התאמה והרחבה. ---

## 1.3.4 מה הפרויקט מחדש ומשפר

### נקודות הייחוד של הפרויקט

| תכונה | מה קיים בשוק | מה עשינו | |-----|-----|-----| | **מניעת Overselling** | רוב המערכות לא מטפלות | MongoDB Transactions אטומיות | | **אבטחת Login** | רק סיסמה | Account Lockout + Rate Limiting + tokenVersion | | **Webhook Reliability** | "fire and forget" | Idempotency + DB Tracking + Token | **Instant Logout** | Retry Logic נשאר תקף | tokenVersion - ביטול מיידי של כל הטוקנים | | **תיעוד** | לרוב חסר | OpenAPI Spec + Postman Collection מלאה |

### תרומת הפרויקט

תיעוד ובדיקות - ברמה הטכנית: מימוש מנגנוני אמינות ואבטחה הרלוונטיים למערכות מסחר - ברמת התחזוקה: קוד מאורגן, מודולרי וניתן להרחבה עתידית ---

# חלק ב' - ליבה טכנית: צד שרת (Backend)

## 1.3.5 דרישות מערכת ופונקציונאליות

### 1.3.5.1 דרישות מערכת (Non-Functional)

Node.js 22.x | Runtime environment | | | **סביבה** | |-----|-----|-----| | פירוט | דרישה | קטגוריה |  
 Latency < 100ms | | **ביצועים** | | MongoDB 8.x | Primary database | | Redis 5.x | Caching & sessions  
 95th percentile | | Throughput 100+ RPS | Requests per second | | Concurrent users 500+ |  
 Graceful shutdown | No lost requests on restart | | Auto- | **שרידות** | | Simultaneous connections  
**עומסים** | | reconnect | DB/Redis connection recovery | | Health checks | Liveness & readiness probes  
 Rate limiting | Per-user & per-IP | | Connection pooling | DB connection reuse | | Response caching |  
 | | Redis-based caching

### 1.3.5.2 דרישות פונקציונאליות

#### מודול Authentication | פעולה | Endpoint | תיאור | |-----|-----|-----| | הרשמה | POST  
 | | /auth/register | יצירת חשבון חדש | | התחברות | POST /auth/login | אימות + JWT token |  
 התחברות עם Google | POST /auth/google | אימות OAuth וקבלת טוקנים | | התנתקות | POST  
 /auth/logout | ביטול כל הטוקנים (tokenVersion) | | רענון טוקן | POST /auth/refresh | קבלת  
 access token חדש | | אימות | GET /auth/verify | בדיקת תקינות הטוקן | | בקשת איפוס סיסמה |  
 POST /auth/forgot-password | יצירת תהליך איפוס סיסמה | | איפוס סיסמה בפועל | POST  
 /auth/reset-password/:token | הגדרת סיסמה חדשה | | פרופיל משתמש | GET/PUT  
 /auth/profile | שליפה ועדכון פרופיל | | שינוי סיסמה | POST /auth/change-password | שינוי  
 סיסמה למשתמש מחובר | #### מודול Products | פעולה | Endpoint | תיאור | |-----|-----|-----|  
 רשימת מוצרים | GET /products | עם סינון וחיפוש | | רשימת קטגוריות | GET  
 /products/categories/list | שליפת קטגוריות מוצרים | | מוצר בודד | GET /products/:id |  
 פרטי מוצר מלאים | #### מודול Cart | פעולה | Endpoint | תיאור | |-----|-----|-----| | צפייה בעגלה  
 | GET /cart | פריטים + סכום | | הוספת פריט | POST /cart/add | עם בדיקת מלאי | | עדכון כמות |  
 PUT /cart/update | עם בדיקת מלאי | | הסרת פריט | DELETE /cart/remove | הסרה מהעגלה |  
 ריקון עגלה | DELETE /cart/clear | הסרת כל הפריטים | #### מודול Orders | פעולה | Endpoint |  
 תיאור | |-----|-----|-----| | יצירת הזמנה | POST /orders | מהעגלה + כתובת | | רשימת הזמנות  
 | GET /orders | הזמנות המשתמש | | הזמנה בודדת | GET /orders/:id | פרטים מלאים | | ביטול |  
 POST /orders/:id/cancel | לפני שילוח בלבד | | מעקב פומבי | GET

### 1.3.6 בעיות צפויות ופתרונות

| בעיה | תיאור | חומרה | השלכות | |-----|-----|-----|-----| | **Race Conditions** | שני לקוחות קונים  
 את המוצר האחרון בו-זמנית | ● קריטי | Overselling, הפסד כספי | | **Brute Force** | ניסיונות login חוזרים  
 לפריצת סיסמה | ● קריטי | גניבת חשבונות | | Stripe | **Webhook Replay** | שולח webhook פעמיים | ●  
 גבוה | חיוב כפול | | **Token Theft** | גניבת JWT token | ● גבוה | התחזות למשתמש | | **DDoS** | הצפת  
 השרת בבקשות | ● בינוני | השבתת השירות |

## 7/65

**Conditions** | 1. Optimistic locking

Pessimistic locking .2

**Brute Force** | | MongoDB Transactions | MongoDB Transactions .3

| 1. CAPTCHA

**Account Lockout + Rate Limit** .2

2FA | | **Webhook** | פשוט יותר מ-CAPTCHA, פשוט יותר מ-Webhook | Account Lockout + Rate Limiting | UX .3

**Replay** | 1. **Idempotency Keys**

auditing | | **Token Theft** | 1. מאפשר, אמיין, פשוט, Event sourcing | Idempotency + DB Tracking .2

Short expiry

**tokenVersion** .2

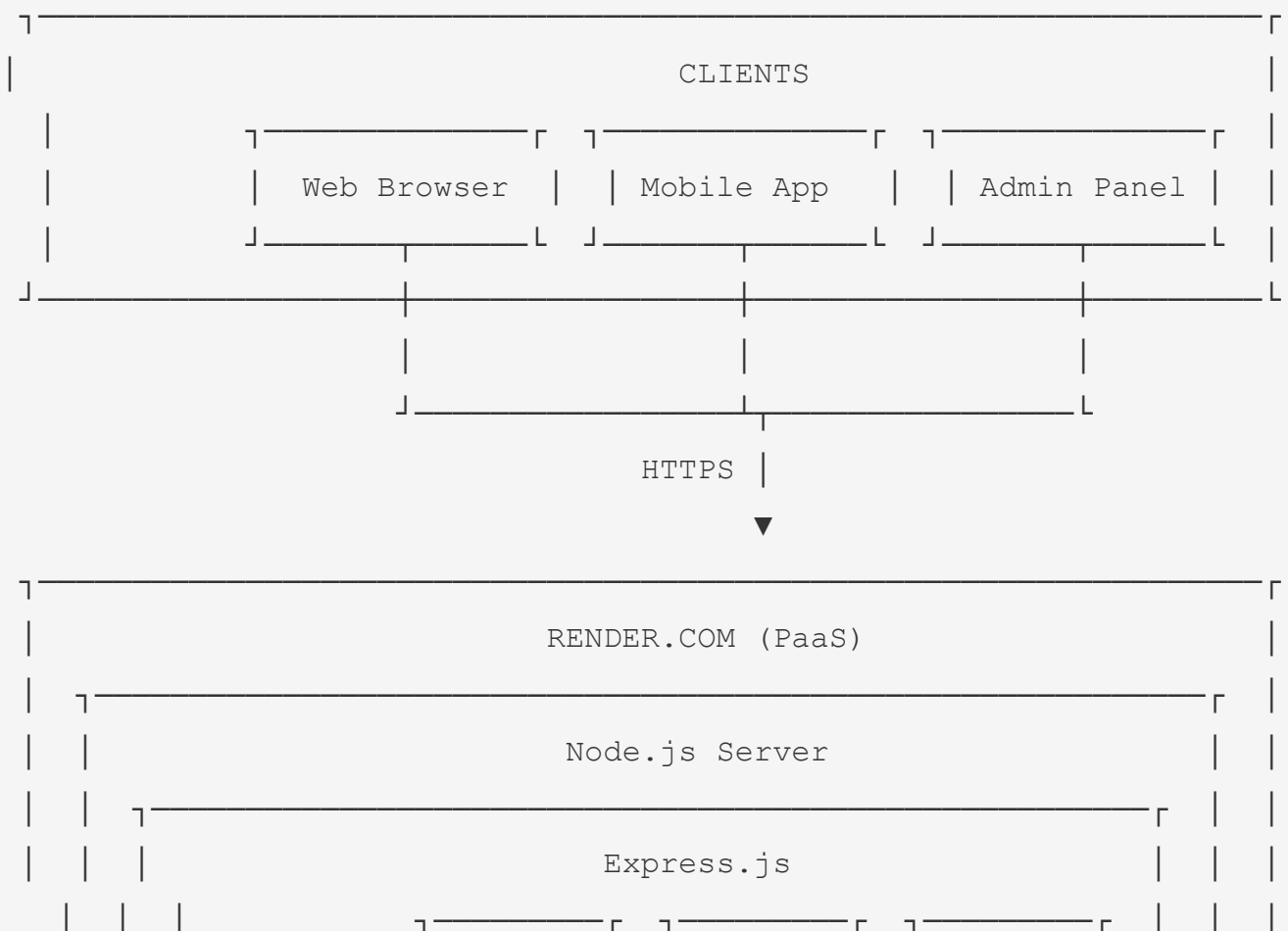
**DDoS** | 1. **Rate Limiting** | | Token blacklist | tokenVersion .3

WAF .2

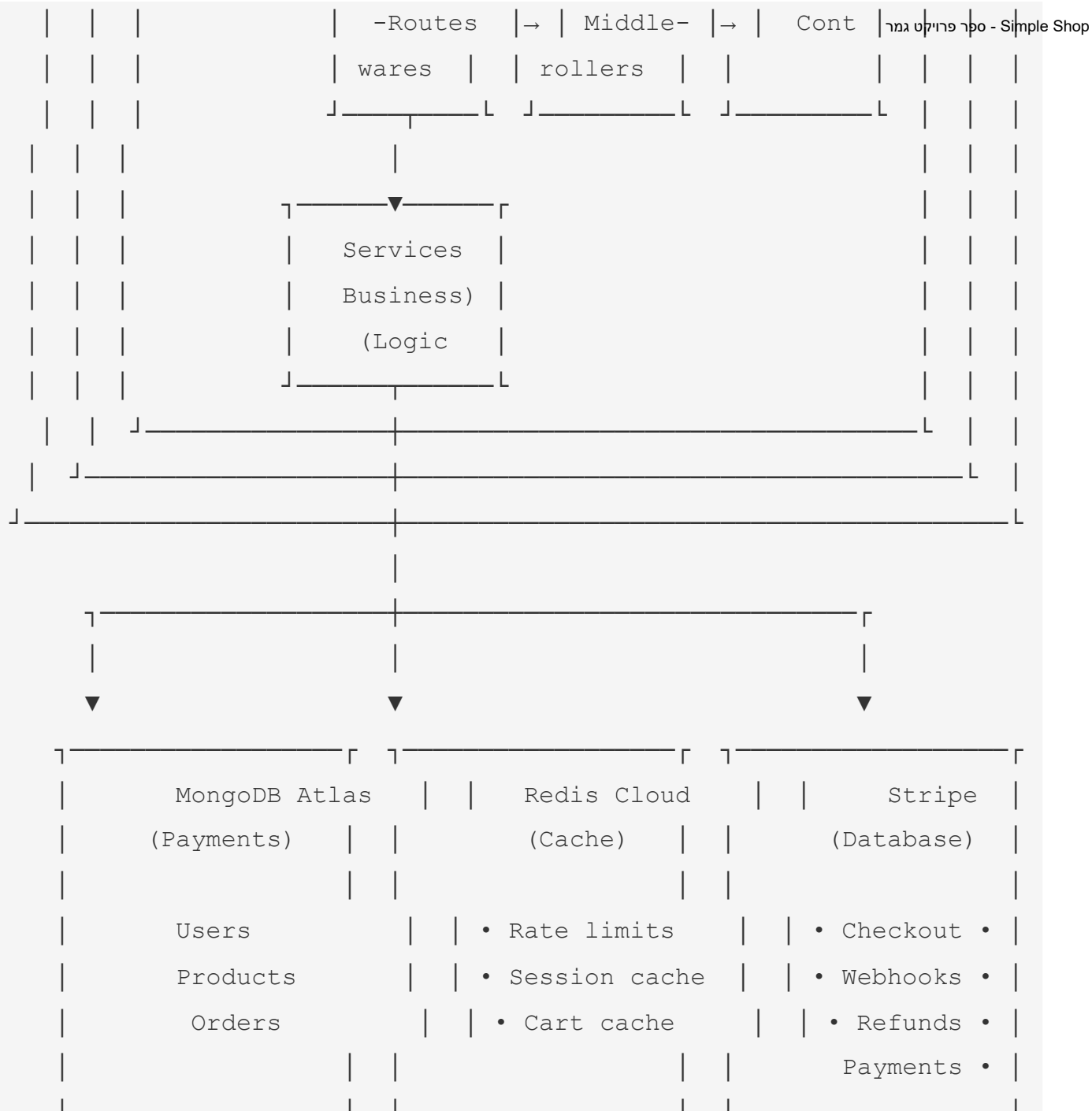
--- | 3. Rate Limiting per IP/User | CDN | פשוט, יעיל, לא דורש שירות חיצוני |

## 1.3.7 פתרון טכנולוגי נבחר

### 1.3.7.1 טופולוגיית הפתרון







## 1.3.7.2 טכנולוגיות בשימוש

שכבה | טכנולוגיה | גרסה | תפקיד | Runtime | Node.js | 22.x | |-----|-----|-----|-----|

JavaScript execution engine | **Framework** | Express.js | 4.19+ | HTTP server & routing | **Language** | TypeScript | 5.x | Type-safe JavaScript | **Database** | MongoDB | 8.x | NoSQL document database | **ODM** | Mongoose | 8.x | MongoDB object modeling | **Cache** | Redis | 5.x | In-memory data store |

**Auth** | JWT | - | Stateless authentication | **Payments** | Stripe | 20.x | Payment processing |

**Validation** | Zod | 3.x | Runtime type validation | **Logging** | Pino | 9.x | Structured JSON logging |

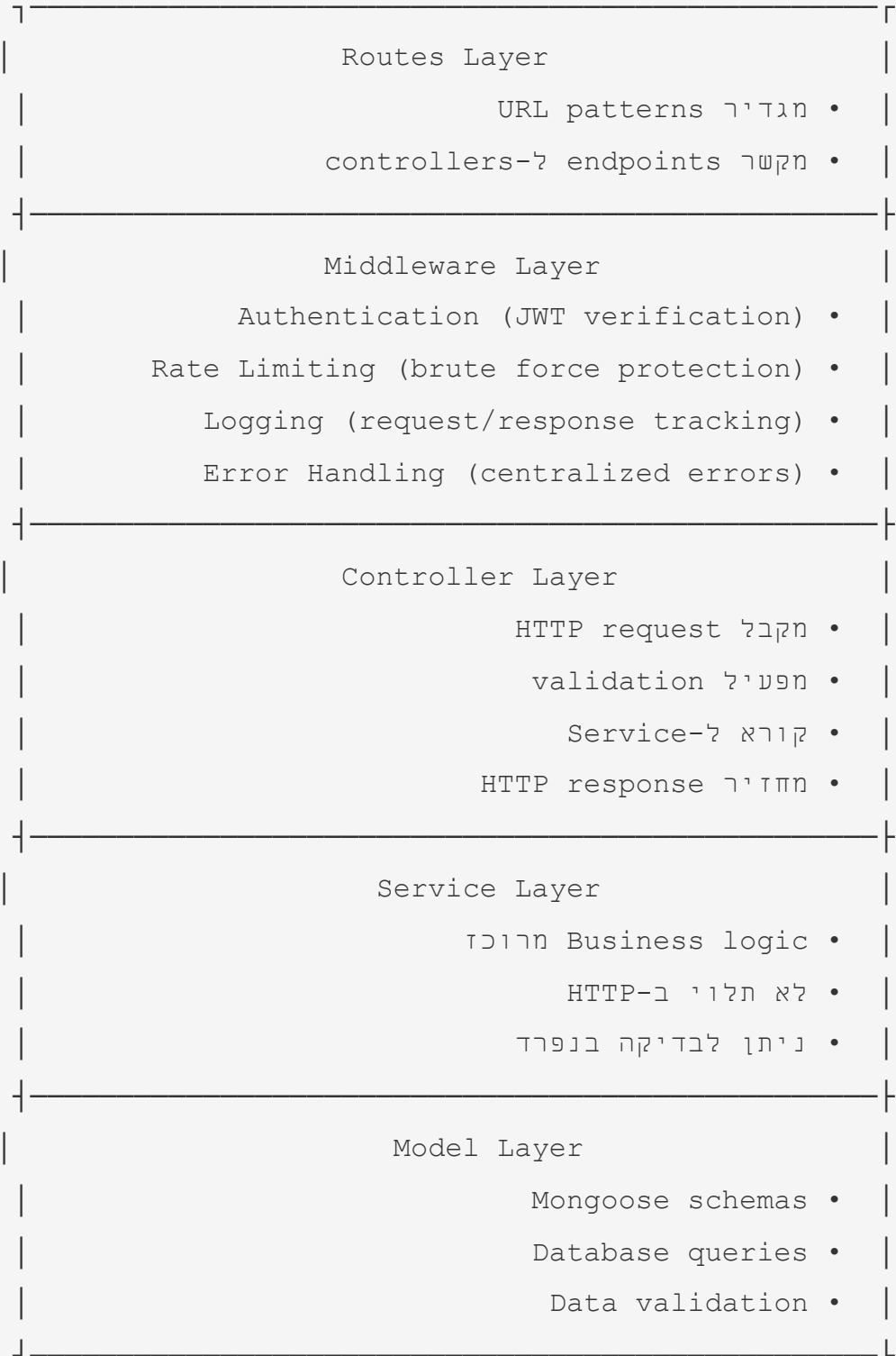
**Testing** | Jest | 29.x | Unit & integration tests | **Metrics** | prom-client | 15.x | Prometheus metrics |

| **API Docs** | Swagger UI + swagger-jsdoc | 5.x / 6.x | OpenAPI documentation

| שפה | שימוש | סיבה | |-----|-----|-----| | **TypeScript** | קוד השרת כולו | Type safety, autocomplete, תחזוקתיות | | **JSON** | Runtime (compiled) | Node.js native support | | **JavaScript ES2023** | Configuration, API | Standard data format | | **YAML** | Docker, CI/CD | Human-readable config

1.3.7.4 תיאור הארכיטקטורה

Pattern: Layered Architecture + MVC



אחראית לדבר אחד | | Testability | אפשר לבדוק כל שכבה בנפרד | | Maintainability | קל למצוא ולתקן

באגים | | Scalability | אפשר להחליף שכבות בלי לשנות אחרות |

### 1.3.7.5 חלוקה לתכניות ומודולים

```
                                /server
                                /src —|
app.ts                          # Express app configuration —| |
    server.ts                  # Server entry point —| |
    swagger.ts                # Inline OpenAPI definition —| |
                                | |
                                routes/                          # URL routing —| |
                                auth.routes.ts —| | |
                                product.routes.ts —| | |
                                cart.routes.ts —| | |
                                order.routes.ts —| | |
                                address.routes.ts —| | |
                                health.routes.ts —| | |
                                metrics.routes.ts —| | |
                                payment.routes.ts —| | |
                                admin.routes.ts —L | |
                                | |
                                controllers/                      # HTTP handlers —| |
                                auth.controller.ts —| | |
                                product.controller.ts —| | |
                                cart.controller.ts —| | |
                                order.controller.ts —| | |
                                address.controller.ts —| | |
                                health.controller.ts —| | |
                                payment.controller.ts —| | |
                                admin.controller.ts —L | |
                                | |
                                services/                        # Business logic —| |
auth.service.ts                # Login, register, tokenVersion —| | |
    googleAuth.service.ts     # Google OAuth verification —| | |
```

|                                |                               |   |  |  |
|--------------------------------|-------------------------------|---|--|--|
| product.service.ts             | # CRUD, filtering             | — |  |  |
| cart.service.ts                | # Cart management             | — |  |  |
| order.service.ts               | # Order creation              | — |  |  |
| address.service.ts             | # Address book management     | — |  |  |
| payment.service.ts             | # Stripe integration          | — |  |  |
| payment-metrics.service.ts     | # Payment and webhook metrics | — |  |  |
| webhook-retry.service.ts       | # Retry failed webhooks       | — |  |  |
| audit-log.service.ts           | # Audit trail helpers         | — |  |  |
| health.service.ts              | # Health helpers              | — |  |  |
| admin.service.ts               | # Stats, management           | — |  |  |
|                                |                               |   |  |  |
| models/                        | # Database schemas            | — |  |  |
|                                | user.model.ts                 | — |  |  |
|                                | product.model.ts              | — |  |  |
|                                | cart.model.ts                 | — |  |  |
|                                | address.model.ts              | — |  |  |
|                                | order.model.ts                | — |  |  |
|                                | payment.model.ts              | — |  |  |
|                                | webhook-event.model.ts        | — |  |  |
|                                | failed-webhook.model.ts       | — |  |  |
|                                | idempotency-key.model.ts      | — |  |  |
|                                | audit-log.model.ts            | — |  |  |
|                                | sequence.model.ts             | — |  |  |
|                                |                               |   |  |  |
| middlewares/                   | # Request processors          | — |  |  |
| auth.middleware.ts             | # JWT verification            | — |  |  |
| rate-limiter.middleware.ts     | # Brute force protection      | — |  |  |
| error.middleware.ts            | # Error handling              | — |  |  |
| logging.middleware.ts          | # Request logging             | — |  |  |
| idempotency.middleware.ts      | # Duplicate prevention        | — |  |  |
| validate.middleware.ts         | # Zod request validation      | — |  |  |
| validateObjectId.middleware.ts | # ObjectId guards             | — |  |  |
| metrics.middleware.ts          | # Request metrics             | — |  |  |
| audit-logging.middleware.ts    | # Audit trail enrichment      | — |  |  |
|                                |                               |   |  |  |
| validators/                    | # Input validation (Zod)      | — |  |  |
|                                | auth.validator.ts             | — |  |  |

|                 |                            |   |   |  |
|-----------------|----------------------------|---|---|--|
|                 | address.validator.ts       | — |   |  |
|                 | order.validator.ts         | — |   |  |
|                 | payment.validator.ts       | — |   |  |
|                 | index.ts                   | — | L |  |
|                 |                            |   |   |  |
| config/         | # Configuration            | — |   |  |
| env.ts          | # Environment variables    | — |   |  |
| db.ts           | # MongoDB connection       | — |   |  |
| redisClient.ts  | # Redis connection         | — |   |  |
| cors.ts         | # CORS settings            | — |   |  |
| constants.ts    | # Shared config constants  | — | L |  |
|                 |                            |   |   |  |
| utils/          | # Utilities                | — |   |  |
| logger.ts       | # Pino logger              | — |   |  |
| asyncHandler.ts | # Async + ApiError helper  | — |   |  |
| response.ts     | # Response helpers         | — |   |  |
| metrics.ts      | # Metrics registry helpers | — | L |  |
|                 |                            |   |   |  |
|                 | /types                     | — |   |  |
|                 | express.ts                 | — |   |  |
|                 | express-session.d.ts       | — | L |  |
|                 |                            |   |   |  |
| tests/          | # Test files               | — | L |  |
|                 | auth.test.ts               | — |   |  |
|                 | auth.test.google.ts        | — |   |  |
|                 | products.test.ts           | — |   |  |
|                 | health.test.ts             | — |   |  |
|                 | order.test.ts              | — |   |  |
|                 | payment-webhook.test.ts    | — |   |  |
|                 | integration.test.ts        | — |   |  |
|                 | performance.test.ts        | — |   |  |
|                 | test-setup.ts              | — | L |  |
|                 |                            |   |   |  |
| docs/           | # Documentation            | — |   |  |
| postman/        | # API collection           | — |   |  |
| scripts/        | # Swagger/export helpers   | — |   |  |
|                 | SETUP.md                   | — |   |  |

```
package.json —|
jest.config.js —|
tsconfig.json —L
```

### 1.3.7.6 סביבת השרת

**Production** | Render.com | PaaS, auto-deploy from | |-----|-----|-----| | שירות | תיאור | |-----|-----|-----| | **Database** | MongoDB Atlas | Managed MongoDB cluster | | **Cache** | Redis Cloud | Managed Redis instance | | **Testing** | לפי הגדרות | | **Development** | Local | Node.js + MongoDB/Redis | | **API Base URL** | Jest against compiled `dist/tests` | | **Docs** | Swagger היא `process.env.API_URL` | | **& Health Endpoints:** - `/api/docs` - Swagger UI - `/api/docs.json` - OpenAPI JSON - `/health` - root health endpoint - `/metrics` - Prometheus scrape endpoint

### 1.3.7.7 ממשק משתמש (GUI)

הממשק מפותח בנפרד על ידי צד הלקוח - ראה [חלק ג'](#).

### 1.3.7.8 ממשקים למערכות אחרות (API)

External APIs 1. **Stripe API** #### תשלומים

```
POST https://api.stripe.com/v1/checkout/sessions
→ יצירת session לתשלום

Webhook: POST /api/payments/webhook
→ קבלת אירועים (payment.succeeded, payment.failed)
```

| Internal API - User / Public Endpoints | Method | Endpoint                  | Auth | Input (main)              | Output (main)                                    | ####                                 |
|----------------------------------------|--------|---------------------------|------|---------------------------|--------------------------------------------------|--------------------------------------|
| Typical errors                         | POST   | /api/auth/register        | No   | { name, email, password } | { success, data: { user, token, refreshToken } } | 409 (email exists), 400 (validation) |
|                                        | POST   | /api/auth/login           | No   | { email, password }       | { success, data: { user, token, refreshToken } } | 401, 423 (account locked), 429       |
|                                        | POST   | /api/auth/google          | No   | { idToken }               | { success, data: { user, token, refreshToken } } | 400, 403, 429                        |
|                                        | POST   | /api/auth/forgot-password | No   | { email }                 | { success, message }                             | 400,                                 |

```
    } | { success, data: { token, refreshToken } } | 400, 401, 429 || GET |
/api/auth/verify | Bearer | header Authorization | { success, data: { user } } |
401 || GET | /api/auth/profile | Bearer | header Authorization | { success, data: {
user } } | 401 || PUT | /api/auth/profile | Bearer | { name?, email? } | { success,
data: { user }, message } | 400, 401 || POST | /api/auth/change-password | Bearer |
{ currentPassword, newPassword, confirmPassword } | { success, message } |
400, 401 || POST | /api/auth/logout | Bearer | header Authorization | { success,
message } | 401 || GET | /api/products | No | query
category,minPrice,maxPrice,search,featured,sort | { success, data:
Product[] } | 400 || GET | /api/products/categories/list | No | - | { success,
data: string[] } | 400 || GET | /api/products/:id | No | path id (ObjectId) | {
success, data: Product } | 400, 404 || GET | /api/cart | Bearer | header
Authorization | { success, data: Cart } | 401 || GET | /api/cart/count | Bearer |
header Authorization | { success, data: { count } } | 401 || POST |
/api/cart/add | Bearer | { productId, quantity } | { success, data: Cart,
message } | 400, 401 || PUT | /api/cart/update | Bearer | { productId, quantity } | {
success, data: Cart, message } | 400, 401, 404 || DELETE | /api/cart/remove | Bearer |
{ productId } | { success, data: Cart, message } | 400, 401, 404 || DELETE |
/api/cart/clear | Bearer | header Authorization | { success, data: { userId,
items: [], total: 0 }, message } | 401, 500 || POST | /api/orders | Bearer +
idempotency | { shippingAddress, billingAddress?, paymentMethod?, notes? } |
{ success, data: { order, payment }, message } | 400, 401 || GET | /api/orders |
Bearer | query status? | { success, data: { orders } } | 401 || GET |
/api/orders/:orderId | Bearer | path orderId | { success, data: { order } } | 400,
401, 404 || POST | /api/orders/:orderId/cancel | Bearer | path orderId | { success,
data: { order }, message } | 400, 401, 404 || GET | /api/orders/track/:orderId | No
| path orderId | { success, data: TrackingResponse } | 400, 404, 429 || GET |
/api/addresses | Bearer | header Authorization | { success, data: Address[] } |
401 || GET | /api/addresses/default | Bearer | header Authorization | { success,
data: Address } | 401, 404 || GET | /api/addresses/:addressId | Bearer | path
addressId | { success, data: Address } | 400, 401, 404 || POST | /api/addresses |
Bearer | { fullName, phone, street, city, postalCode, country, isDefault? } |
{ success, data: Address, message } | 400, 401 || PUT |
```

```

/api/addresses/:addressId | Bearer | partial address fields | { success, data: Address, message } | 400, 401, 404 || DELETE | /api/addresses/:addressId | Bearer |
path addressId | 204 No Content | 400, 401, 404 || POST |
/api/addresses/:addressId/set-default | Bearer | path addressId | { success,
data: Address, message } | 400, 401, 404 || POST | /api/payments/create-intent |
Bearer | { orderId } | { success, data: { payment, status, clientSecret,
checkoutUrl }, message } | 400, 401, 404 || POST | /api/payments/checkout | Bearer | {
orderId } | { success, data: { payment, status, clientSecret, checkoutUrl
}, message } | 400, 401, 404 || GET | /api/payments/:orderId/status | Bearer | path
orderId | { success, data: { orderPaymentStatus, paymentStatus, ... },
message } | 400, 401, 404 || POST | /api/payments/webhook | No (Stripe signature) | raw body
+ stripe-signature header | { received: true } | 400 || GET | /api/health | No | - |
{ success, data: { status, mongodb, redis, webhooks, uptime } } | 500 || GET |
/api/health/ping | No | - | { success, message: "pong", data: { time } } | 500 ||
GET | /api/metrics/payment | Bearer | query lastN? | { success, data:
PaymentMetrics } | 401 || GET | /api/metrics/webhook | Bearer | query lastN? | {
success, data: WebhookMetrics } | 401 || GET | /api/metrics/all | Bearer | - | {
success, data: ExportedMetrics } | 401 | ##### Internal API - Admin Only Endpoints
requireAdmin. | Method | Endpoint | Input (main) | דורשים */api/admin/ תחת
Output (main) | Typical errors | |-----|-----|-----|-----|-----| | GET |
/api/admin/stats/summary | - | { success, data: { stats } } | 401, 403 || GET |
/api/admin/products | query includeInactive? | { success, data: { products }
} | 401, 403 || POST | /api/admin/products | product payload | { success, data: {
product } } | 400, 401, 403 || PUT | /api/admin/products/:id | path id, patch payload |
{ success, data: { product } } | 400, 401, 403, 404 || DELETE |
/api/admin/products/:id | path id | { success, data: { product }, message } |
400, 401, 403, 404 || GET | /api/admin/users | query page?,limit? | { success, data:
{ users, total, page, pages } } | 401, 403 || PUT | /api/admin/users/:id/role |
path id, body { role } | { success, data: { user } } | 400, 401, 403, 404 || GET |
/api/admin/orders | query status?,userId? | { success, data: { orders } } |
401, 403 || PUT | /api/admin/orders/:id/status | path id, body { status, message?
- רוב ה- } | { success, data: { order } } | 400, 401, 403, 404 | ##### Response Format
endpoints מחזירים מעטפת בסגנון הבא, אך לא כל ה-routes זהים לחלוטין. לדוגמה, health/ מחזיר
.code - errors error responses עשויים לכלול גם success/status/timestamp

```



```

        Success //
    }

    ,success: true
    ,{ ... } :data
    "message?: "Optional message
    {

        Error //
    }

    ,success: false
    ,"message: "Error description
    ,[...] :?errors
    "code?: "ERROR_CODE
    {

```

### 1.3.7.9 שימוש בחבילות תוכנה

express | ^4.19 || |-----|-----|-----| | תפקיד | גרסה | חבילה | Production Dependencies #####  
 HTTP server framework || ioredis | ^5.4 | Redis client || stripe | ^20.1 | Payment processing ||  
 jsonwebtoken | ^9.0 | JWT creation & verification || google-auth-library | ^10.x | Google  
 OAuth token verification || bcryptjs | ^3.0 | Password hashing || helmet | ^8.1 | Security  
 headers || cors | ^2.8 | Cross-origin support || zod | ^3.25 | Schema validation || pino | ^9.x |  
 חבילה | Structured logging || prom-client | ^15.0 | Prometheus metrics | ##### Dev Dependencies  
 mongoose | ^8.x | MongoDB ODM (used by the app) || |-----|-----|-----| | תפקיד | גרסה |  
 typescript | ^5.4 | Type checking || jest | ^29.7 | Testing framework || supertest | ^7.1 |  
 HTTP testing || ts-node | ^10.9 | TypeScript execution || ts-node-dev | ^2.0 | Development  
 | auto-restart || @types/\* | various | TypeScript definitions

### 1.3.7.10 פונקציות מרכזיות

AuthService.login() || |-----|-----|-----| | פונקציה | קובץ | תיאור |  
 JWT | | אימות משתמש, בדיקת נעילה, יצירת | server/src/services/auth.service.ts  
 tokenVersion | AuthService.logout() | server/src/services/auth.service.ts  
 לביטול כל הטוקנים | | AuthService.register() |  
 | | יצירת משתמש, hash סיסמה | server/src/services/auth.service.ts

יצירת | `OrderService.createOrder()` | `server/src/services/order.service.ts` | `StripeSession` | `Stripe` | `Simple Shop`

הזמנה מעגלה ||| `PaymentService.createPaymentIntent()`

יצירת | `Stripe session` | `server/src/services/payment.service.ts`

| `PaymentService.handleWebhook()` | `server/src/services/payment.service.ts`

עיבוד | `webhook, idempotency, amount verification` | `CartService.addToCart()`

|| `server/src/services/cart.service.ts` | הוספה לעגלה עם בדיקת מלאי ||

| `ProductService.getProducts()` | `server/src/services/product.service.ts`

שליפה עם סינון ||| `AdminService.getStatsSummary()`

| `server/src/services/admin.service.ts` | סטטיסטיקות מכירות ומלאי ||

| `AuthMiddleware.requireAuth()` | `server/src/middlewares/auth.middleware.ts`

אימות | `JWT + tokenVersion` | `authRateLimiter` | `server/src/middlewares/rate-`

--- | `brute force` | `limiter.middleware.ts` | הגנה מ-

## 1.3.8 מבני נתונים וארגון קבצים

### 1.3.8.1 פירוט מבני הנתונים

User Schema #####

```
typescript
    }
    ,id: ObjectId_
    email: string, // unique, indexed
word?: string, // bcrypt hashed, omitted for Google-only users
    ,name: string
    ,role: "user" | "admin
    ,isActive: boolean
    ,lastUpdated?: Date
tokenVersion: number, // ++ on logout, invalidates all tokens
    ,failedLoginAttempts: number
    ,lockedUntil?: Date | null
    ,lastLogin?: Date
    ,resetPasswordToken?: string
    ,resetPasswordExpires?: Date
    ,googleId?: string | null
    ,avatar?: string | null
    ,createdAt: Date
```

{

## Product Schema #####

```

                                typescript
                                }
                                ,id: ObjectId_
                                sku: string,                // unique
                                ,name: string
                                ,description: string
                                ,price: number
                                ,stock: number
                                category: string,           // indexed
                                ,Valid categories: accessories, audio, displays, laptops //
                                smart-home, smartphones, streaming, tablets, wearables //
                                ,image: string
                                ,featured: boolean
                                ,rating: number
                                isActive: boolean,          // soft delete flag
                                ,createdAt: Date
                                updatedAt: Date
                                {

```

## Cart Schema #####

```

                                typescript
                                }
                                ,id: ObjectId_
                                userId: ObjectId,           // ref: User, unique
                                ]] :items
                                product: ObjectId,         // ref: Product
                                ,quantity: number
                                lockedPrice: number | null
                                ,[{
                                ,total: number
                                ,createdAt: Date

```

{

## Address Schema #####

```

        typescript
    }

    ,id: ObjectId_
    ,user: string
    ,fullName: string
    ,phone: string
    ,street: string
    ,city: string
    ,postalCode: string
    ,country: string
    ,isDefault: boolean
    ,createdAt: Date
    updatedAt: Date

    {

```

## Order Schema #####

```

        typescript
    }

    ,id: ObjectId_

    orderNumber: string,          // unique, format: ORD-YYYYMMDD-XXX
    user: string,                  // ref: User
    ]] :items
    ,product: string
    name: string,                  // snapshot at order time
    price: number,                 // snapshot at order time
    ,quantity: number
    image?: string
    ,[{
    ,totalAmount: number

    nt" | "confirmed" | "processing" | "shipped" | "delivered" | "cancelled
    , "paymentStatus: "pending" | "paid" | "failed" | "refunded
    , "paymentMethod: "creditcard" | "paypal" | "cashion_delivery" | "stripe

```

```
ingHistory: Array<{ status: string; timestamp: Date; message?: string }
                                ,estimatedDelivery?: Date
                                ,notes?: string
                                ,paymentIntentId?: string
                                ,paymentIntentStripeId?: string
                                ,paymentVerifiedAt?: Date
                                ,"paymentProvider?: "stripe" | "paypal" | "mock
                                ,fulfilled: boolean
                                ,fulfilledAt?: Date
                                ,createdAt: Date
                                updatedAt: Date
```

### Payment Schema #####

```

                                typescript
                                }
                                ,id: ObjectId_
                                order: string,           // ref: Order
                                user: string,             // ref: User
                                ,amount: number
                                ,currency: string
" | "requires action" | "succeeded" | "failed" | "refunded" | "canceled

```

```

        ,provider: string
        ,providerPaymentId?: string
        ,paymentIntentId?: string
        ,clientSecret?: string
        ,checkoutUrl?: string
        ,meta?: Record
        ,createdAt: Date
        ,updatedAt: Date
    }

```

#### WebhookEvent Schema (Idempotency) ####

```

        typescript
    }

    ,id: ObjectId_

eventId: string, // unique, from Stripe
    ,provider: "stripe" | "paypal"
    ,eventType: string
    ,processedAt: Date
    ,metadata?: any
    ,createdAt: Date
    ,updatedAt: Date
    {

```

#### FailedWebhook Schema ####

```

        typescript
    }

    ,id: ObjectId_
    ,eventId: string
    ,eventType: string
    ,provider: "stripe" | "paypal"
    ,payload: any
    ,error: string
    ,retryCount: number
    ,maxRetries: number
    ,nextRetryAt: Date
    ,lastAttemptAt: Date

```

```

, "status": "pending" | "retrying" | "failed" | "succeeded" | "simple Shop
, createdAt: Date
  updatedAt: Date
}

```

## 1.3.8.2 שיטת האחסון

**Primary Data** | MongoDB Atlas | |-----|-----|-----| | סיבה | סוג מידע | טכנולוגיה |  
Document-based, scalable, transactions | | **Sessions/Cache** | Redis Cloud | Fast, in-memory, TTL  
URLs | | **Logs** | Pino → stdout | רק | לא מאחסנים קבצים, support | | **Files/Images** | External URLs  
| Render.com collects automatically

## 1.3.8.3 מנגנוני התאוששות

Startup Recovery Strategy #####

```

typescript
    } try
    ;()await connectMongo
    } catch (err) {
    rn({ err }, "Continuing without Mongo connection for health readiness")
    {
    } try
    ;()await connectRedis
    } catch (err) {
    rn({ err }, "Continuing without Redis connection for health readiness")
    {

```

Transaction Rollback #####

```

typescript
    Atomic stock reduction - all or nothing //
    ;()const session = await mongoose.startSession
    ;()session.startTransaction
    } try
    )await Product.updateOne
    ,{ id: productId, stock: { $gte: quantity }_ }

```

```

        { session }

        ; (

;()await session.commitTransaction

        } catch (error) {

;()await session.abortTransaction

        ;throw error

    {

```

### Webhook Retry Logic #####

```

                                typescript

    Idempotency - prevent duplicate processing //
;const existing = await WebhookEvent.findOne({ eventId })

                                } if (existing)

                                ;return { status: 'already_processed' }

                                {

                                Process and save eventId //
;await WebhookEvent.create({ eventId, processedAt: new Date() })

```

— — —

### 1.3.9 תרשימי מערכת

### Use Case Diagram 1.3.9.1





צפייה במוצרים

הרשמה / התחברות

User

(משתמש)

ניהול עגלה

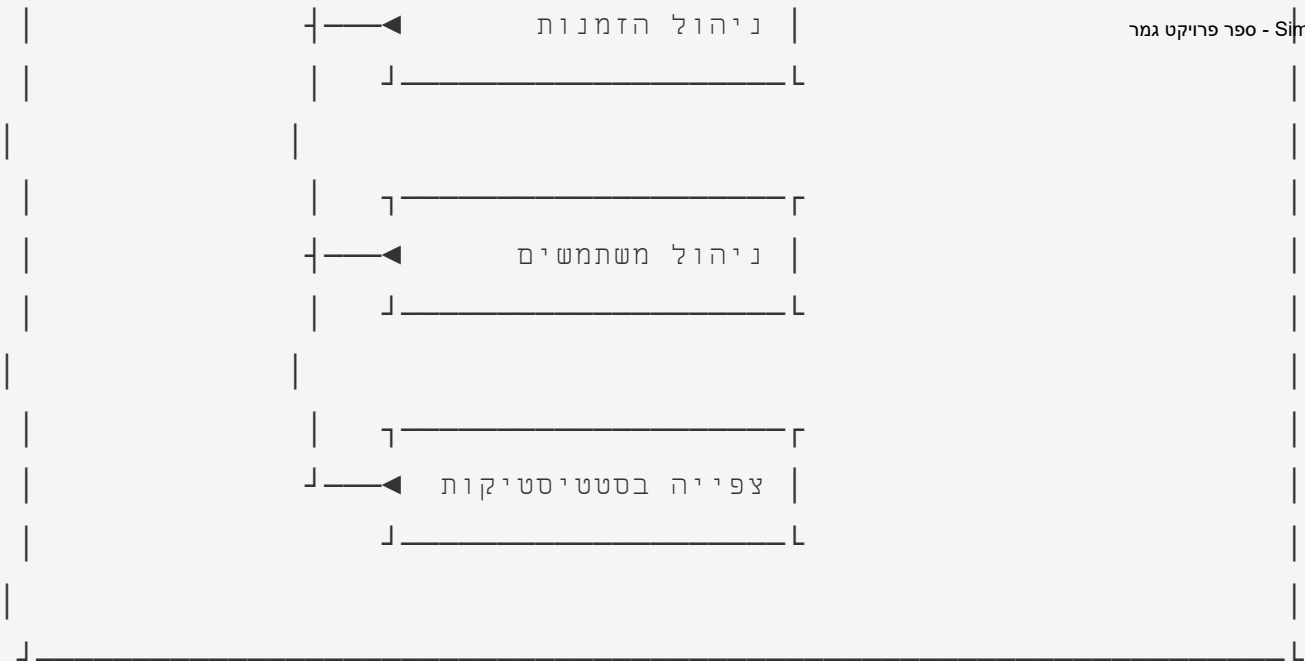
ביצוע הזמנה

תשלום ב-Stripe

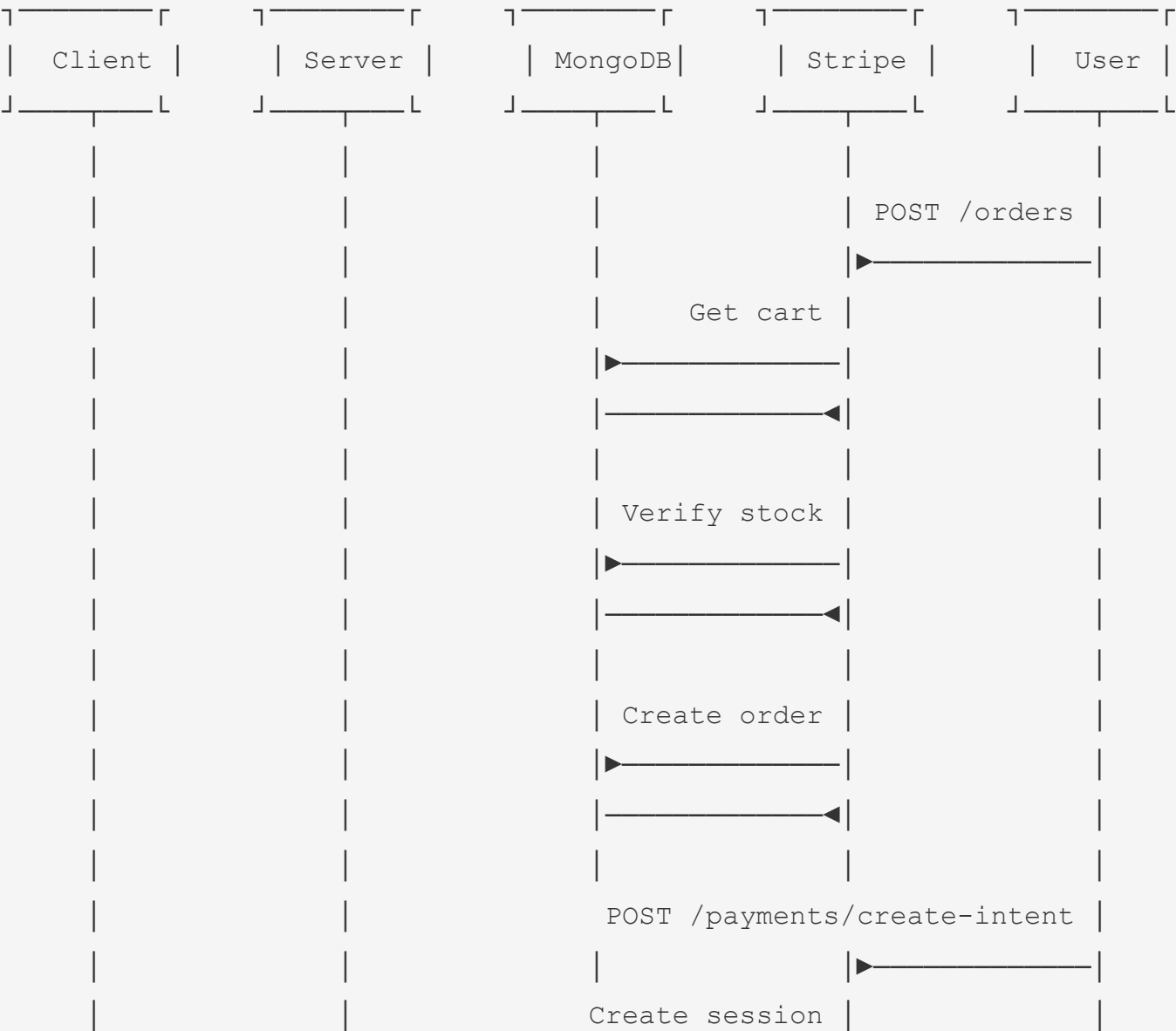
צפייה בהזמנות

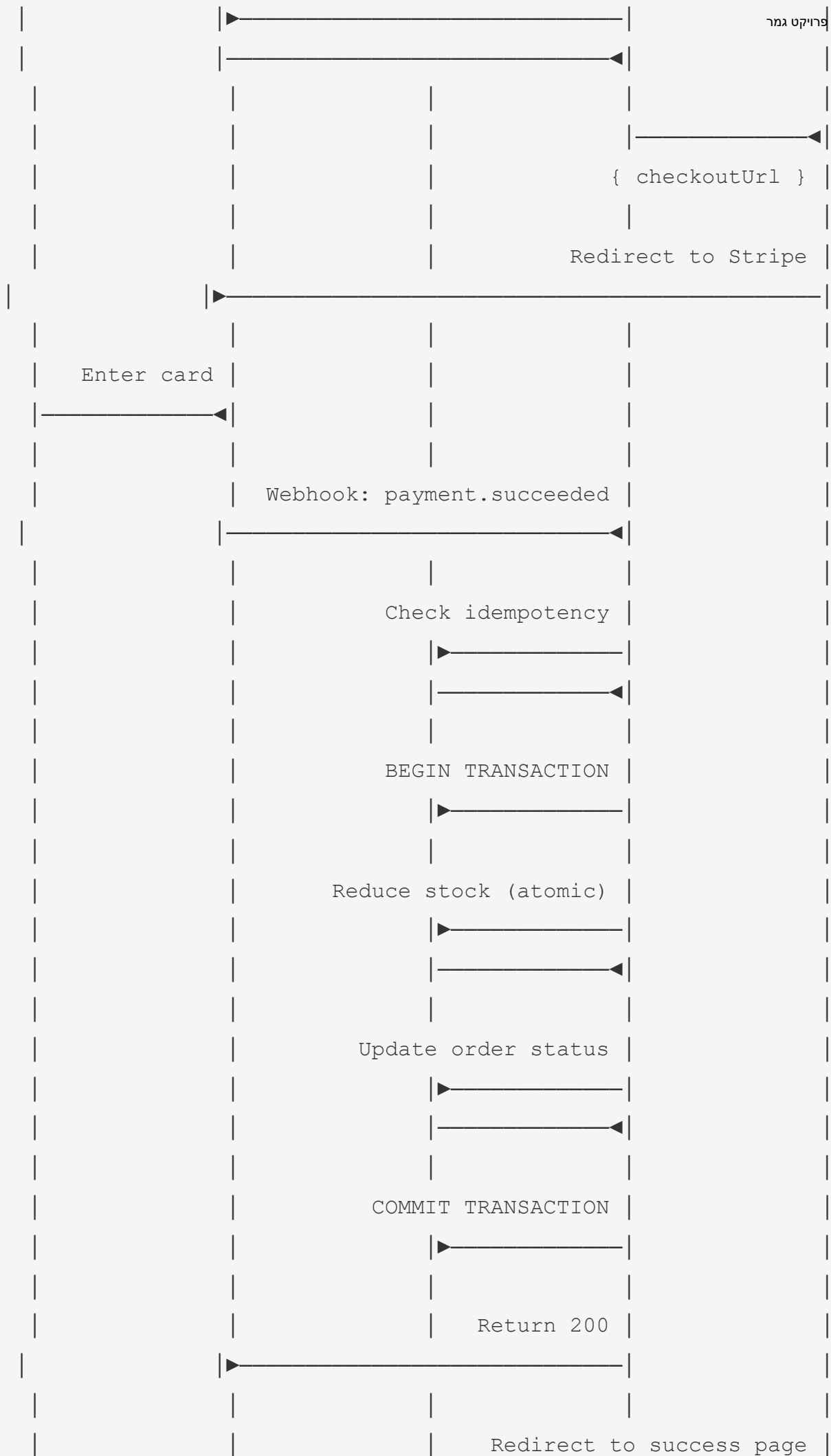
ניהול פרופיל

ניהול מוצרים

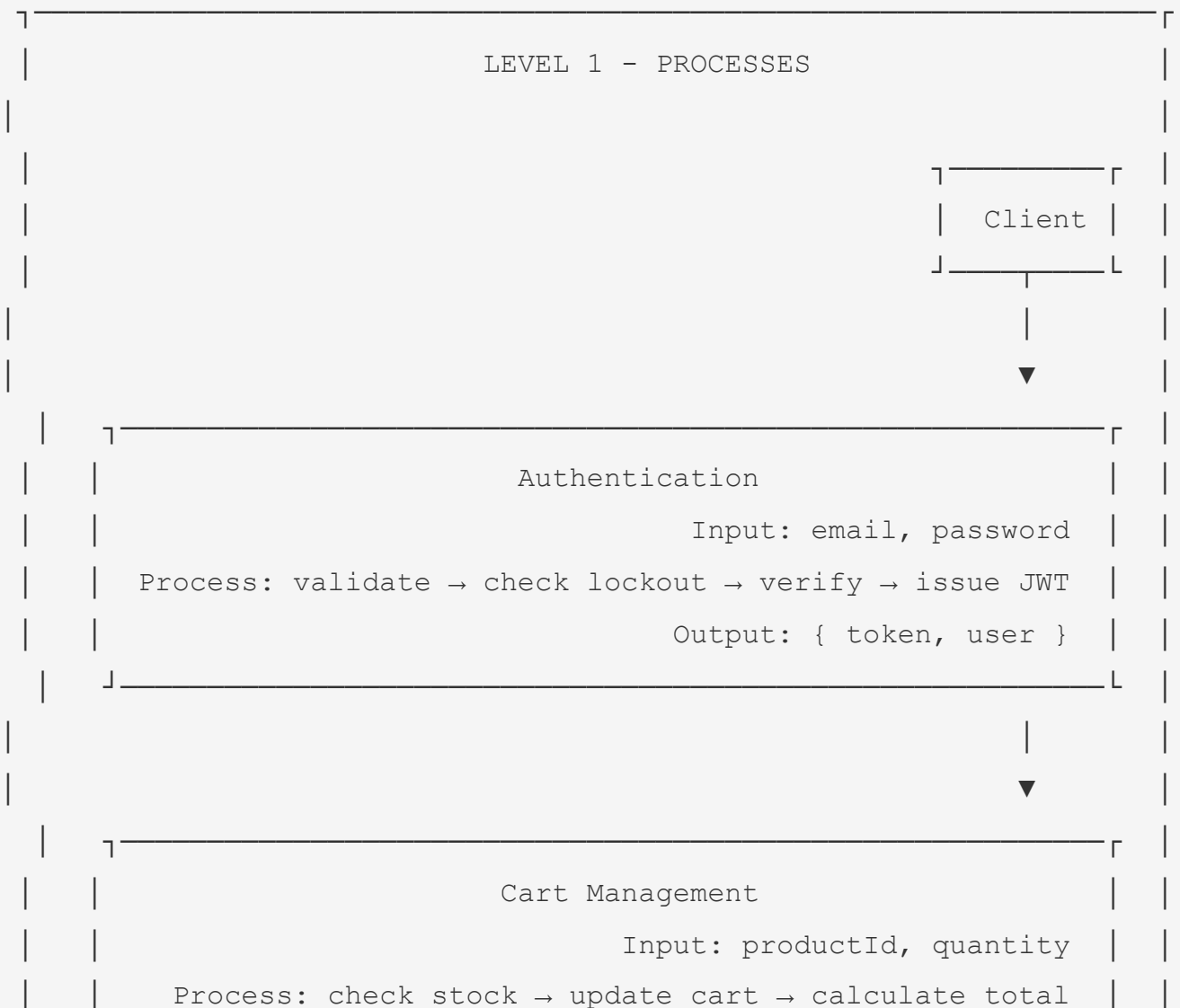
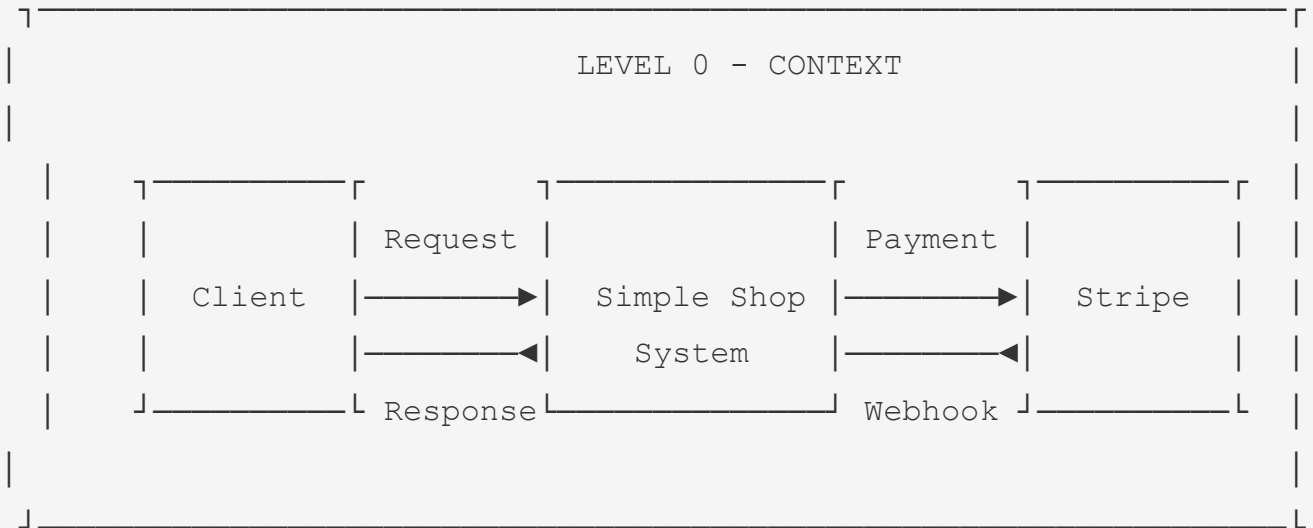


Sequence Diagram - Checkout Flow 1.3.9.2



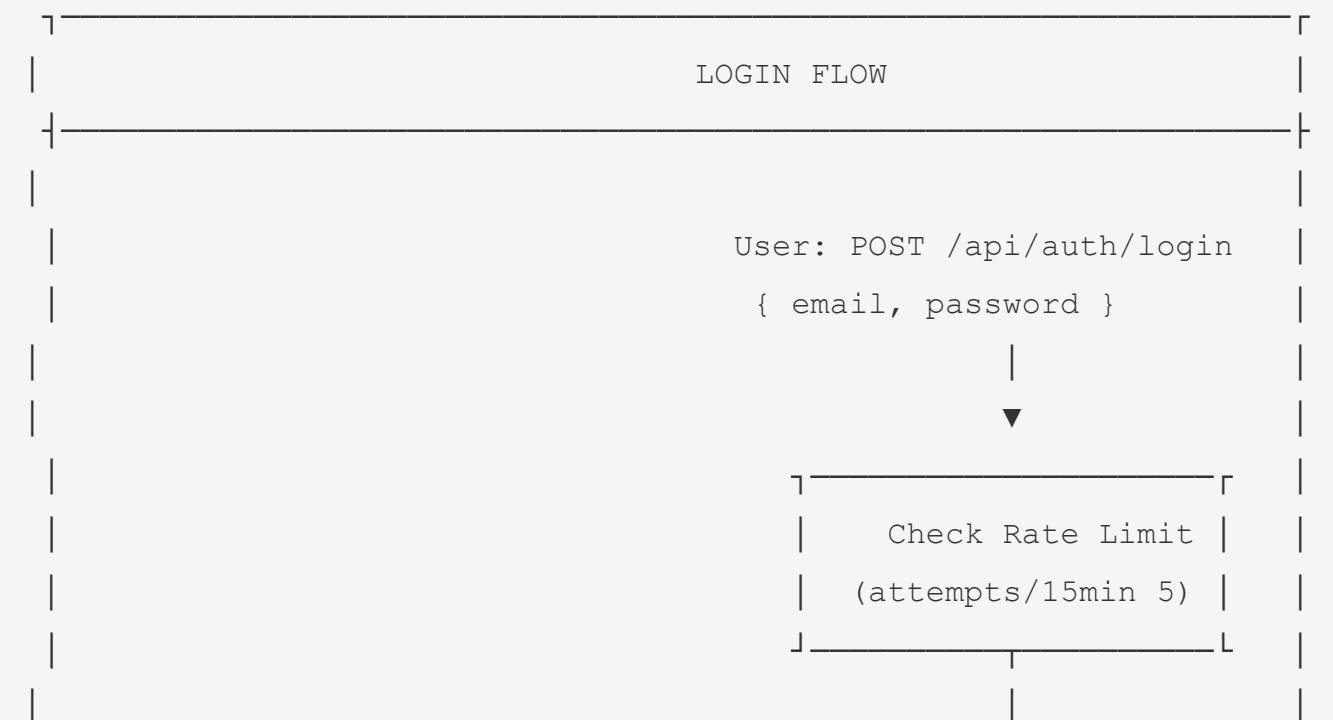


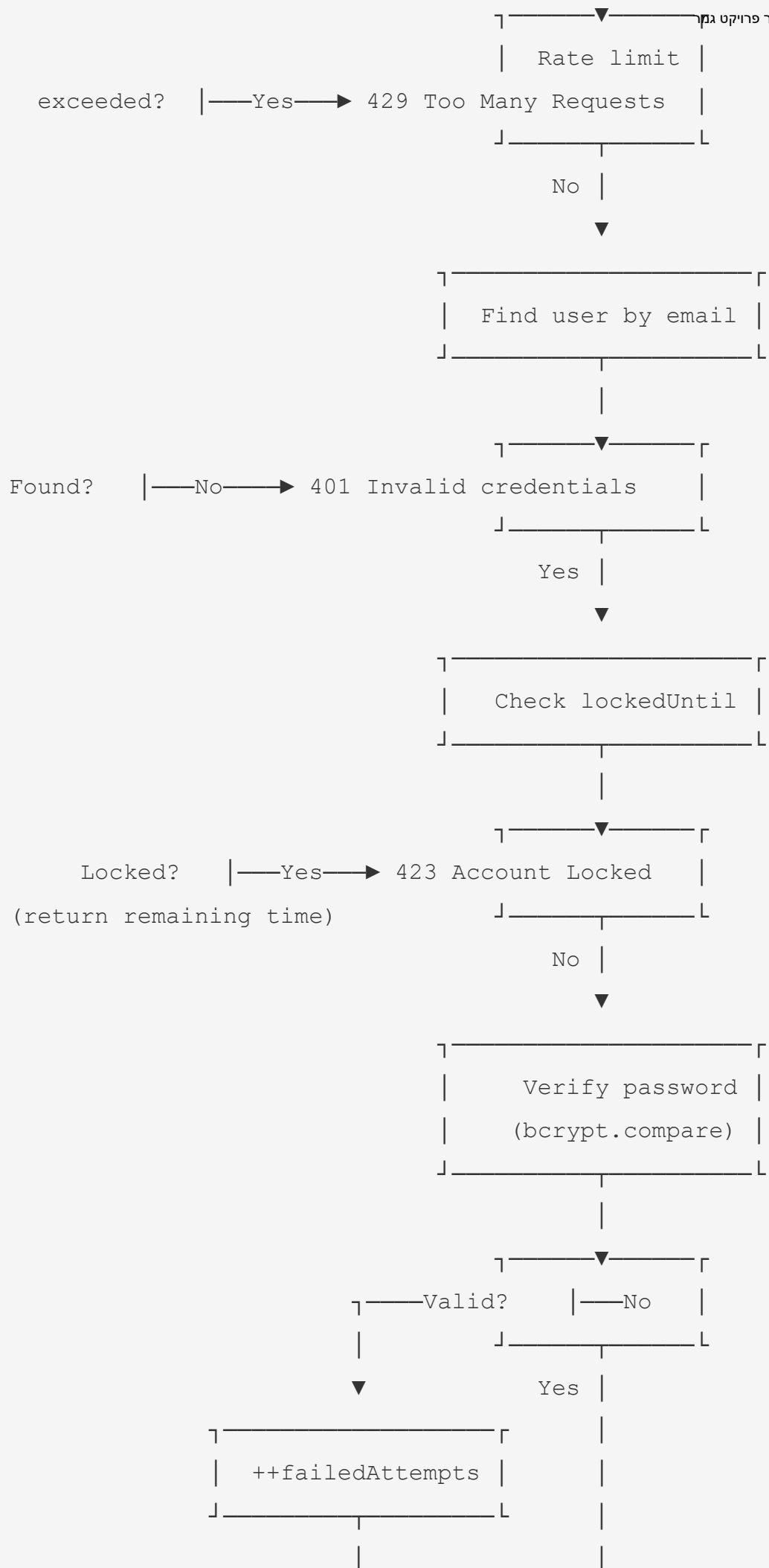
### Data Flow Diagram 1.3.9.3

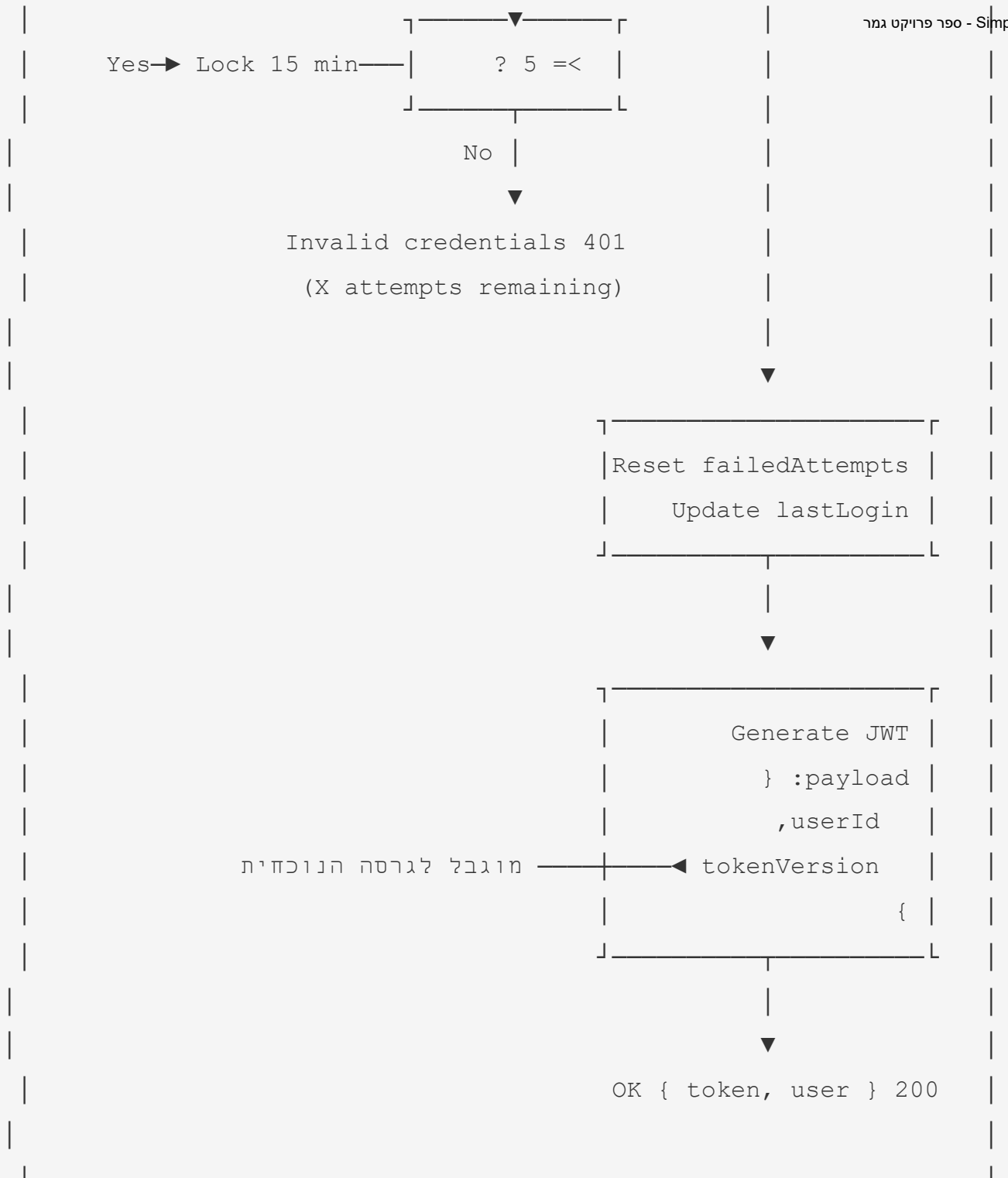




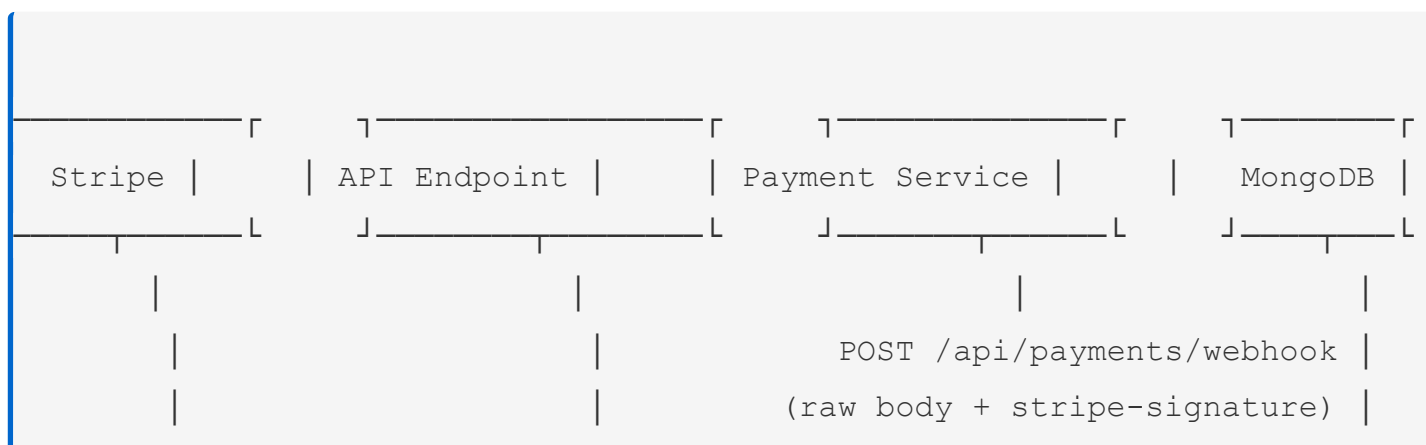
## 1.3.9.4 תרשים עם Login tokenVersion

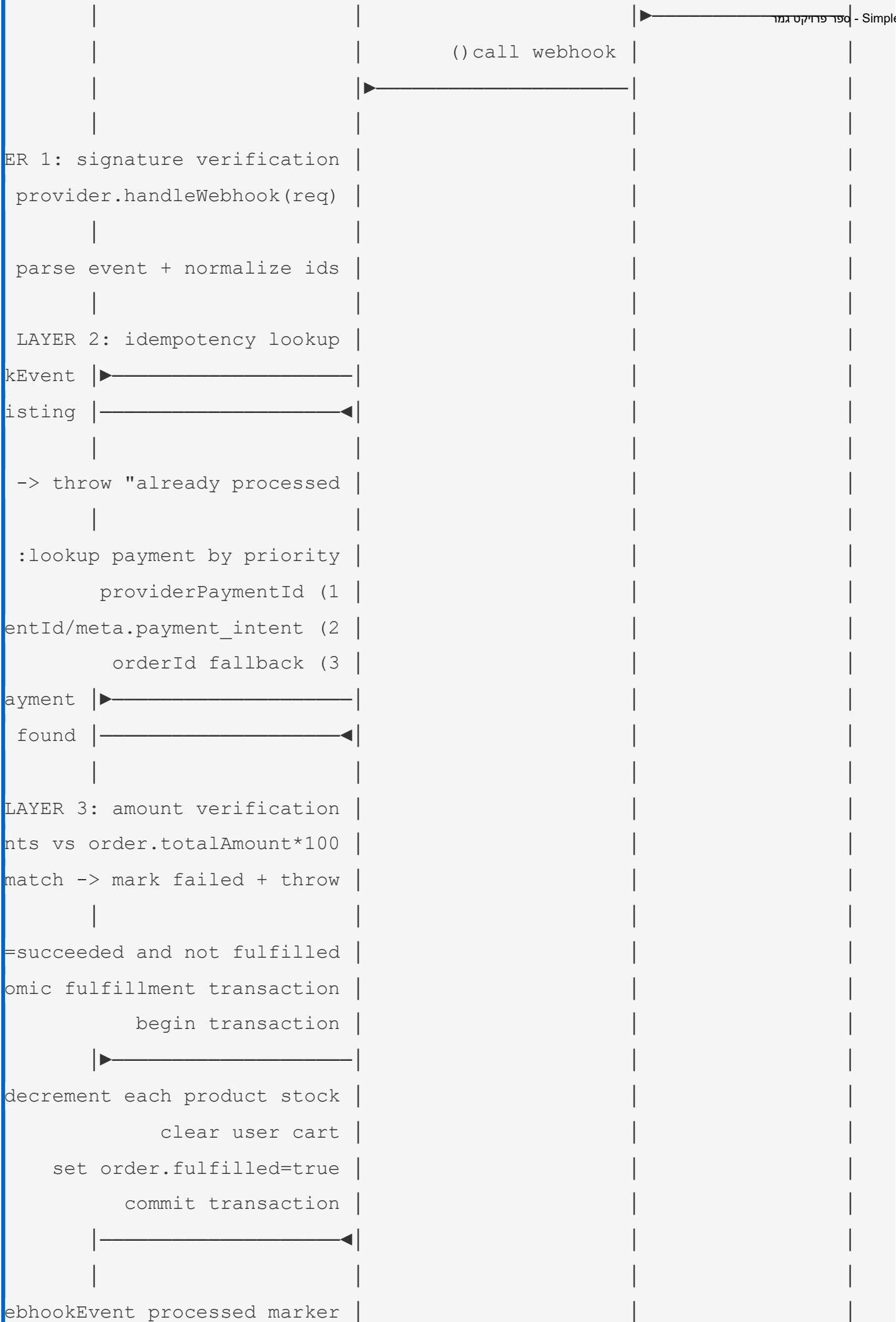




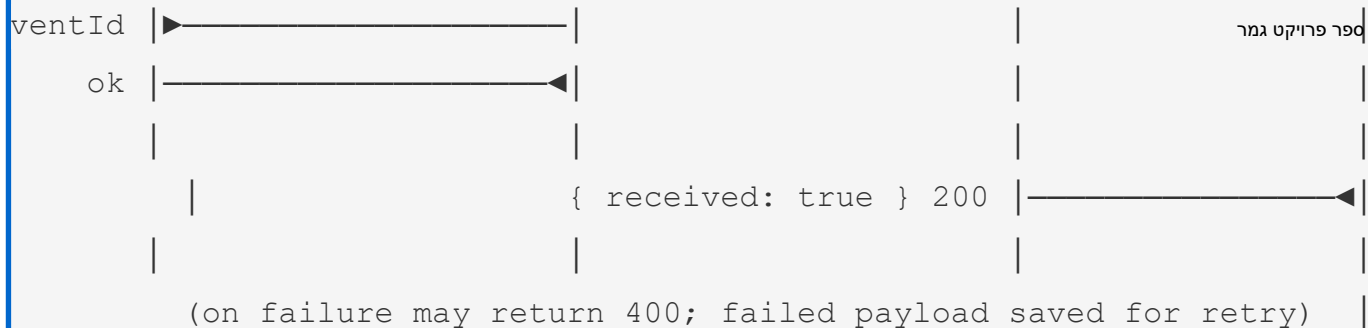


### Sequence Diagram - Detailed Webhook Security Flow 1.3.9.5

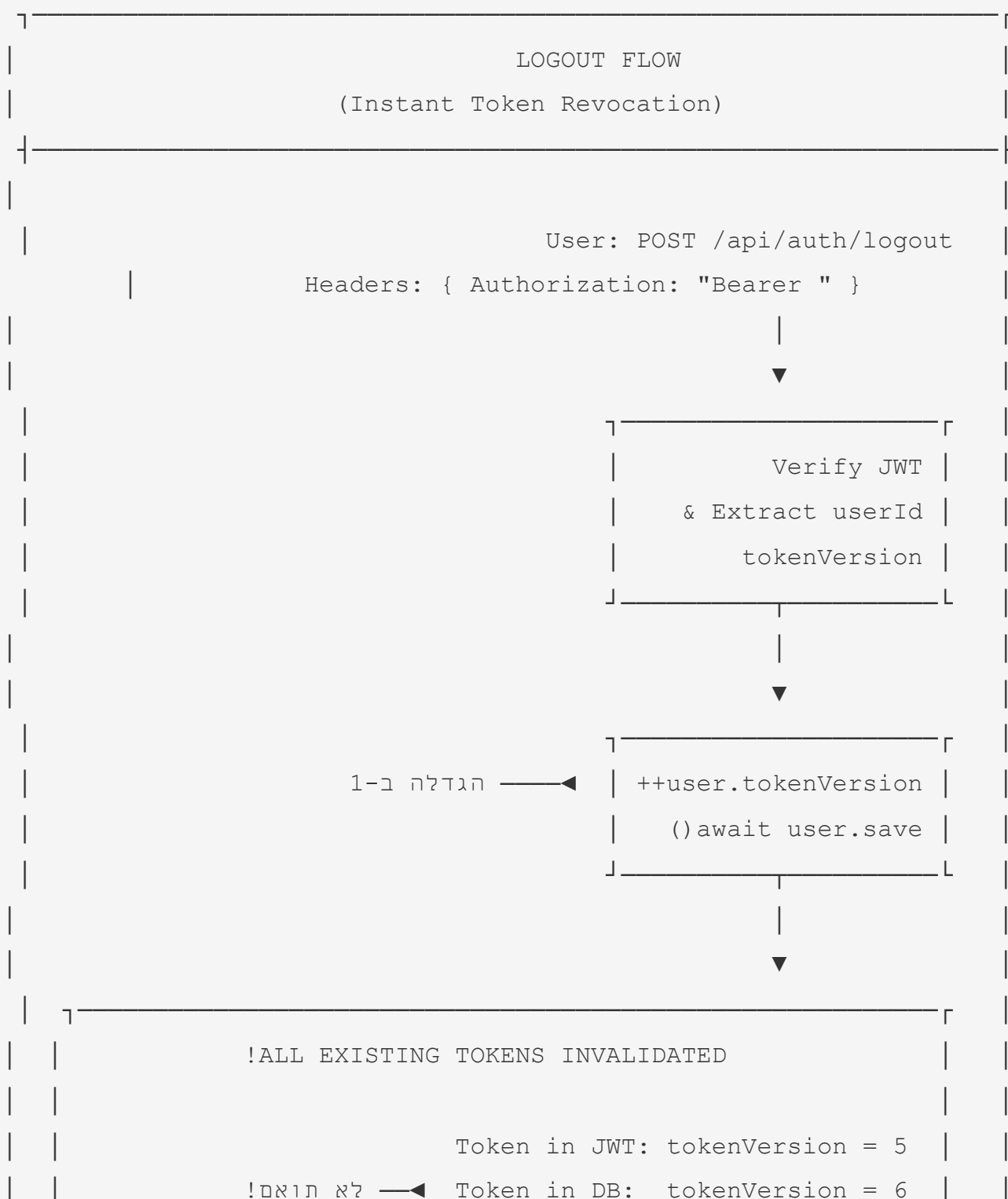








## 1.3.9.5 תרשים Logout עם tokenVersion





---

## 1.3.10 מרכיב אלגוריתמי

### 1.3.10.1 בעיות ופתרונות אלגוריתמיים

#### בעיה 1: Race Condition במכירת מוצרים **הבעיה:**

| Time                                                                                   | User A         | User B         | Stock |
|----------------------------------------------------------------------------------------|----------------|----------------|-------|
| T1                                                                                     | Read stock (1) |                | 1     |
| T2                                                                                     |                | Read stock (1) | 1     |
| T3                                                                                     | Buy (stock--)  |                | 0     |
|  T4 |                | Buy (stock--)  | -1    |

### הפתרון: MongoDB Atomic Update

```

typescript
// במקום:
const product = await Product.findById(id)
if (product.stock >= quantity)
!product.stock -= quantity; //  Not atomic
await product.save()

// עשינו:
const result = await Product.findOneAndUpdate
  
```

```

stock: { $gte: quantity } // Condition in query
    }, {
    }

    inc: { stock: -quantity } // Atomic decrement$
    }, {
    { new: true }
    }; (

    } if (!result)
    ;throw new Error('Insufficient stock')
    {

```

**סיבוכיות:** single atomic operation -  $O(1)$  --- #### בעיה 2: ביטול מידי של כל הטוקנים (Logout) **הבעיה:**  
 משתמש מתנתק, אבל JWT tokens שכבר הופקו עדיין תקפים עד שיפוגו. **פתרונות אפשריים:** | פתרון |  
 יתרון | חיסרון | |-----|-----|-----| | Token Blacklist | פשוט | DB lookup בכל בקשה | | Short Expiry |  
 פשוט | UX רע (login תכוף) | | **tokenVersion** | מהיר, ללא DB lookup נוסף | צריך לשמור version | **הפתרון**  
**שבחרנו: tokenVersion**

```

typescript
// ב-User Schema
tokenVersion: { type: Number, default: 0 }

// ב-JWT payload Login - כולל את הגרסה:
const token = jwt.sign
  ,userId: user._id
  tokenVersion: user.tokenVersion // e.g., 5
  ;(secret ,{

// ב-Logout - מגדילים את הגרסה:
user.tokenVersion += 1; // Now 6
;()await user.save

// ב-Auth Middleware - בודקים התאמה:
;const decoded = jwt.verify(token, secret)
;const user = await User.findById(decoded.userId)

```

```

} if (decoded.tokenVersion !== user.tokenVersion)
  throw new Error('Token revoked'); // 5 !== 6
{

```

**סיבוכיות:**  $O(1)$  - comparison operation --- #### בעיה 3: Webhook Replay Attack **הבעיה:** Stripe שולח webhook פעמיים (network issue, retry) → חיוב כפול / stock כפול **הפתרון:** **Idempotency Tracking + Duplicate Guard**

```

typescript
} async function handleWebhook(req: Request)
const result = await provider.handleWebhook(req); // signature verified

const existingEvent = await WebhookEventModel.findOne
  ,eventId: result.providerPaymentId
  ,provider: provider.name
  ; ({

  } if (existingEvent)
w Error(Webhook event ${result.providerPaymentId} already processed)
{

...payment lookup + amount verification + fulfillment flow //

  })await WebhookEventModel.create
  ,eventId: result.providerPaymentId
  ,eventType: result.status
  ,provider: provider.name
  ,metadata: result.raw
  ; ({
{

```

2- `PaymentController.webhook` כפילות מזהה מוחזרת כ-200 עם הודעת `Duplicate event` `ignored`, כך ש-Stripe לא ימשיך לנסות לעבד את אותו אירוע שוב ושוב. **סיבוכיות:**  $O(1)$  lookup +  $O(1)$  insert (עם אינדקס על `eventId/provider`) ---

## 1.3.10.2 איסוף מידע וסטטיסטיקות

Response Time | prom-client | |-----|-----|-----| | שימוש | מקור | Metric | נאספים | Metrics ##### Simple Shop

זיהוי | Failed Logins | Auth service | | התראות | Error Rate | Error middleware | | זיהוי

התקפות | | Payment Success Rate | Payment | | התראת מלאי נמוך | Stock Levels | Product model | |

Admin Stats Endpoint ##### service | בריאות עסקית |

```
typescript
GET /api/admin/stats/summary //
    }
    } :sales
    (delivered) סה"כ מכירות //      ,total: 12345.67
    deliveredCount: 45
    ,{
    } :orders
    הזמנות פתוחות //      ,open: 8
    today: 3
    ,{
    } :inventory
    ,lowStockCount: 5
    stock < 5 מוצרים עם //      ] :lowStockProducts
    { id, name, stock_ }
    ,[
    activeProducts: 50
    ,{
    } :users
    total: 150
    {
    {
```

Logging Format (Structured) #####

```
json
    }
    , "level": "info"
    , "time": 1709308800000
    , "service": "PaymentService"
    , "method": "handleWebhook"
    , "eventId": "evt_1abc123"
    , "eventType": "checkout.session.completed"
```

```
, "orderId": "ORD-20260301-001" - Simple Shop
    , amount": 199.99"
    , duration_ms": 45"
    "status": "success"
  }
```

---

## 1.3.11 אבטחת מידע

### אזורים הדורשים הגנה

**Authentication** | Brute force, Credential stuffing | Rate | |-----|-----|-----| | אזור | איומים | הגנות |  
limit, Account lockout, bcrypt | | **Sessions** | Token theft, Session fixation | JWT, tokenVersion, HTTPS  
only | | **API** | Injection, XSS, CSRF | Zod validation, Helmet, CORS | | **Database** | SQL injection, Data  
leak | Mongoose ODM, select: false | | **Payments** | Webhook spoofing, Replay | HMAC verification,  
| Idempotency

### מנגנוני ההגנה המיושמים

#### Password Security .1 ####

```
typescript
Hash with bcrypt (12 rounds = ~250ms) //
;const hash = await bcrypt.hash(password, 12)

Verify //
;const isValid = await bcrypt.compare(password, hash)
```

#### JWT with tokenVersion .2 ####

```
typescript
Token payload //
}

, "userId": "507f1f77bcf86cd799439011
  tokenVersion: 5, // Must match DB
  , iat: 1709308800
    exp: 1709312400 // 1 hour
```

```

        Verification includes version check //
    } if (decoded.tokenVersion !== user.tokenVersion)
        ;throw new UnauthorizedError('Token revoked')
    {

```

### Rate Limiting .3 ####

```

                                typescript
Auth endpoints: 5 requests per 15 minutes per user //
                                })const authLimiter = rateLimit
                                    ,windowMs: 15 60 1000
                                    ,max: 5
                                keyGenerator: (req) => req.body.email || req.ip
                                    ;({

General API: 100 requests per minute //
                                })const apiLimiter = rateLimit
                                    ,windowMs: 60 * 1000
                                    max: 100
                                    ;({

```

### Account Lockout .4 ####

```

                                typescript
                                } if (user.failedLoginAttempts >= 5)
;user.lockedUntil = new Date(Date.now() + 15 60 1000)
                                ;throw new Error('Account locked for 15 minutes')
                                {

```

### Webhook Signature Verification .5 ####

```

                                typescript
;const signature = req.headers['stripe-signature']
                                )const event = stripe.webhooks.constructEvent
                                    ,req.body
                                    ,signature

```

; (

Throws if signature invalid //

## 5 תרחישי התקפה ותגובה

# | התקפה | תגובה | קוד שגיאה | |-----|-----|-----|---| | 1 | Brute force login (100 ניסיונות) | Account locked, logged, IP rate limited | 429/423 | | 2 | SQL Injection in email field | Zod validation rejects, query parameterized | 400 | | 3 | Stolen JWT token used | tokenVersion mismatch after logout | 401 | | 4 | Fake Stripe webhook | HMAC signature verification fails | 400 | | 5 | Same webhook sent --- --- | twice | Idempotency check blocks duplicate | 200 (skip)



# חלק ג' - ליבה טכנית: צד לקוח (Frontend)

## 2. צד לקוח (Frontend)

### 2.1 סקירה כללית

צד הלקוח הוא אפליקציית **React** מודרנית המבוססת על Vite ו-TypeScript, ומיועדת לתקשר עם ה-API של השרת. בבדיקת הקוד הקיים בפועל ב- `client/src` נמצא כי קיימת תצורת בסיס טובה שניתנת להרחבה בשלבים: - קיימים קבצי הליבה של האפליקציה (`App.tsx`, `main.tsx`, `store.ts`, `hooks.ts`), `api/axios.ts` ורכיבי UI מרכזיים. - קיימים רכיבי `admin` ורכיבי עגלה/אימות בסיסיים. - ב- `App.tsx` וב- `store.ts` יש ייבואים לתיקיות `Features`, `types`, `services`, `pages` וכן `/components/Header`. - חלק מהמודולים עדיין נמצאים בתהליך הרחבה, בהתאם לחלוקת העבודה בין המפתחים. | היבט | מימוש | |-----|-----| | **ביצועים** | Vite כבסיס build עם תשתית מתאימה להמשך הרחבה | | **עיצוב** | CSS + MUI (Material UI) מקומי | | **Routing** | Redux Toolkit + React useState | **State** | `react-router-dom v7` דרך `App.tsx` | **שילוב** | `Axios instance` עם `Bearer token` מתוך `localStorage`

### 2.2 טכנולוגיות

טכנולוגיה | גרסה | תפקיד | |-----|-----|-----| | **React** | 19.x | UI Library | | **Vite** | 7.x | Build tool | | **TypeScript** | 5.8.x | Type safety | | **Redux Toolkit** | 2.x | Global state management | | **Material UI** | 7.x | UI component library | | **React Router DOM** | 7.x | Client-side routing | | **Axios** | 1.x | HTTP client | | **React Toastify** | 11.x | Notifications

### 2.3 מבנה קבצים

```

/client
/src —|
  user/cart state + ראשי App.tsx # Router —| |
  main.tsx # Entry point + Redux Provider —| |
  store.ts # Redux store —| |
  hooks.ts # Typed redux hooks —| |
  styles.css # Global styles —| |
  # קובץ נתוני מוצרים מקומי product.json —| |

```

|        |                                         |                    |                   |
|--------|-----------------------------------------|--------------------|-------------------|
|        | vite-env.d.ts                           | # Vite types       | 100 - Simple Shop |
|        |                                         | /api               |                   |
| ios.ts | # Axios instance + Authorization header |                    |                   |
|        |                                         | /components        |                   |
|        | AddToCartButton.tsx                     |                    |                   |
|        | Card.tsx                                |                    |                   |
|        | ClearAllBtn.tsx                         |                    |                   |
|        | GoogleConnectBtn.tsx                    |                    |                   |
|        | HomePageBtn.tsx                         |                    |                   |
|        | LogoutButton.tsx                        |                    |                   |
|        | QuantityUpdate.tsx                      |                    |                   |
|        | RemoveFromCartButton.tsx                |                    |                   |
|        |                                         | /admin             |                   |
|        | AddProduct.tsx                          |                    |                   |
|        | OrdersTable.tsx                         |                    |                   |
|        | ProductsTable.tsx                       |                    |                   |
|        | StatsCards.tsx                          |                    |                   |
|        | UsersTable.tsx                          |                    |                   |
|        |                                         | /Footer            |                   |
|        | Footer.tsx                              |                    |                   |
|        | Footer.css                              |                    |                   |
|        |                                         | /assets            |                   |
|        | react.svg                               |                    |                   |
|        |                                         | package.json       |                   |
|        |                                         | package-lock.json  |                   |
|        |                                         | eslint.config.js   |                   |
|        |                                         | vite.config.ts     |                   |
|        |                                         | tsconfig.json      |                   |
|        |                                         | tsconfig.app.json  |                   |
|        |                                         | tsconfig.node.json |                   |

**הערה תכנונית:** קיימים ייבואים לשכבות נוספות (, Features/, types/, services/, pages/), כחלק מכיוון ההרחבה של הלקוח. (/components/Header

קומפוננטה | תפקיד | AddToCartButton | |-----|-----|-----| | Props | | הוספת מוצר לעגלה  
 דרך addToCart() ואז ריענון העגלה עם | product, setCart | getCart()  
 QuantityUpdate | כפתורי הגדלה/הקטנה של כמות | | quantity, onIncrease, onDecrease |  
 RemoveFromCartButton | הסרת מוצר מהעגלה דרך, | product, | removeFromCart()  
 LogoutButton | | onRemoved | | התנתקות ועדכון | GoogleConnectBtn | | setUser | | user state |  
 Google Identity Services, התחברות עם Google, ושמירת token | refreshToken |  
 Card | | setUser | | | הצגת מוצר והעברת הפעולה ל- | product, setCart | AddToCartButton |  
 ProductsTable | טבלת ניהול מוצרים לאדמין | ללא props משמעותיים | | StatsCards | | תצוגת  
 נתוני dashboard למנהל | ללא props משמעותיים | | Footer | | אזור תחתון גלובלי של האפליקציה | ללא  
 | props

## State-1 Hooks 2.5

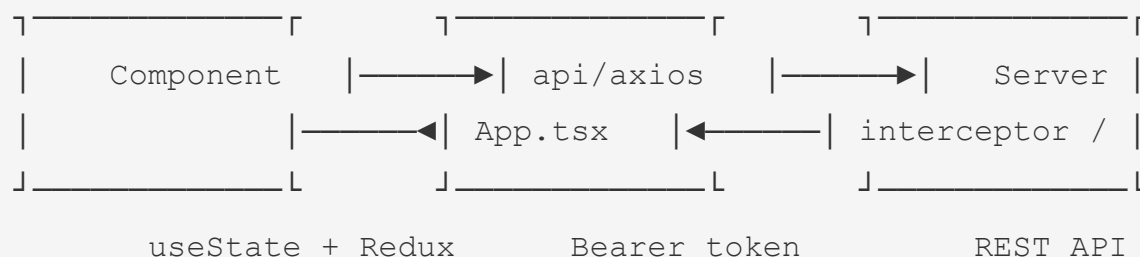
```
typescript
Redux typed hooks //
;()const dispatch = useAppDispatch
;const cart = useAppSelector((state) => state.cart)

Redux store current shape //
    } :reducer
    ,cart: cartReducer
    counter: counterReducer
    {

Local React state in App.tsx //
;const [cart, setCart] = useState(null)
;const [user, setUser] = useState(null)
```

בפועל, App.tsx מבצע useEffect לטעינת משתמש מחובר (verifyUser) ולטעינת העגלה (getCart). **הערה חשובה:** ה-store מפנה ל- Features/cartSlice |  
 Features/counterSlice, מה שמעיד על כיוון ברור לארגון state מתקדם בצד לקוח.

## Data Flow 2.6



בקוד שנמצא כיום: - `api/axios.ts` מגדיר `baseUrl` קשיח ל-`request interceptor`. - `Render` מוסיף `Authorization: Bearer` מתוך `localStorage`. - שכבת `/services` מתוכנתת ומיובאת ב- `App.tsx`, אך קבצי השירות עצמם אינם קיימים כרגע. - `GoogleConnectBtn.tsx` משתמש גם ב- `refreshToken` ושומר אותו ב- `localStorage` לאחר התחברות `Google`.

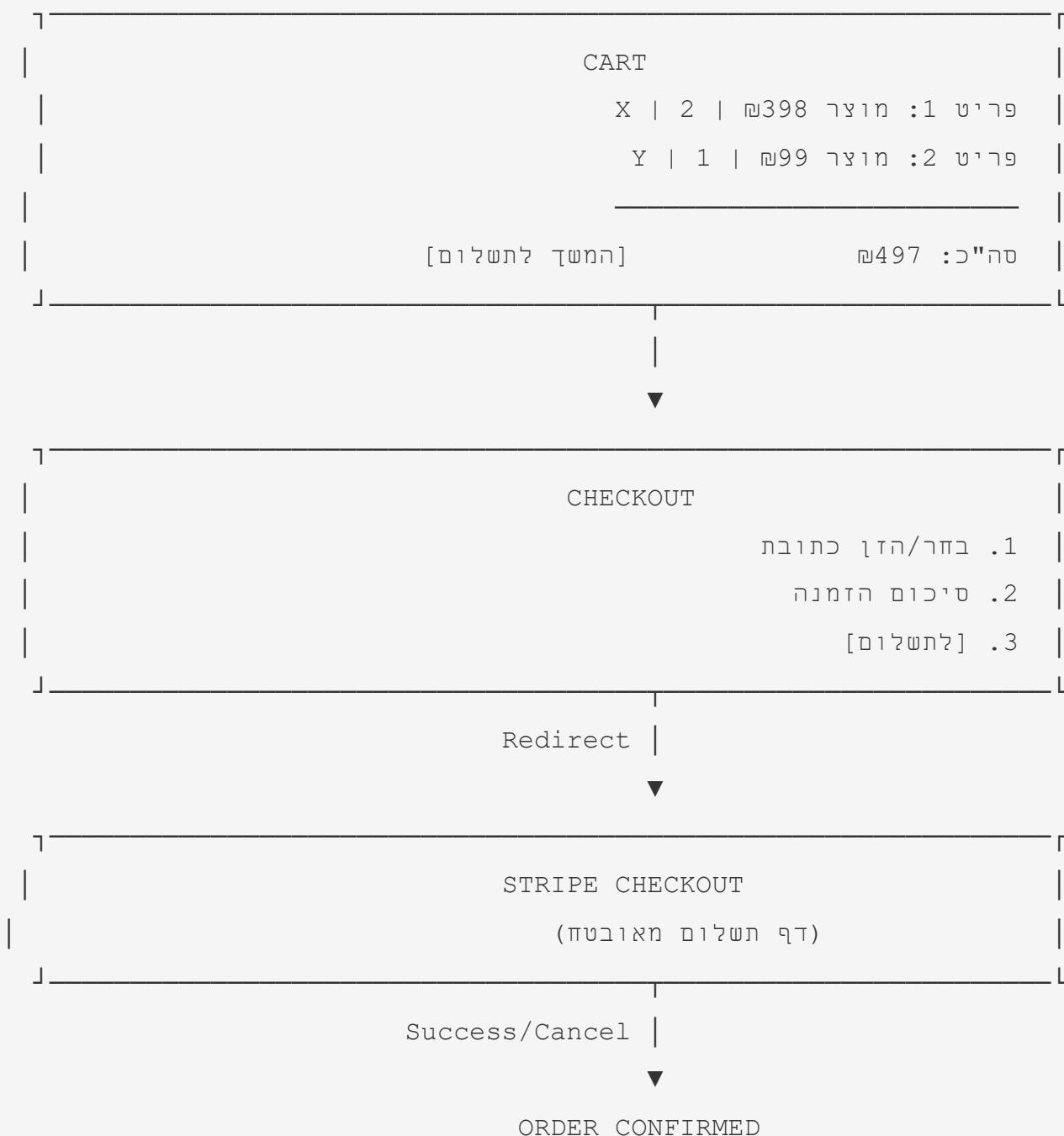
## 2.7 דפי האפליקציה

הנתיבים הבאים מוגדרים ב- `App.tsx` ומשקפים את מבנה ה-`flow` שתוכנן בצד לקוח: דף | נתיב | סטטוס  
 תכנון | | Home | |-----|-----|----|  
 router | | router | | Cart | /cart | מוגדר ב-  
 router | | Payment | /payment | מוגדר ב- router | | Login | /login | מוגדר ב- Register | /register  
 router | | Admin | /admin | מוגדר ב- router | | Checkout | /checkout | מוגדר ב-  
 ProductsTable | | Admin Product Create | מוגדר ומחובר ל- Admin Products | /admin/products  
 | | AddProduct | מוגדר ומחובר ל- | /admin/products/new

## 2.8 תרשים זרימת משתמש

התרשים הבא מתאר את ה-`flow` העסקי שתוכנן בצד הלקוח בהתאם לתצורת ה-`routing` הקיימת.





## 2.9 אבטחה בצד לקוח

נושא | מימוש | |-----|-----| | token | Token Storage נשמר ב- `localStorage` ; בזרימת Google  
 נשמר גם Authorization Header | | `refreshToken` | מתווסף אוטומטית ב- | | `axios interceptor`  
 Protected Routes | מומלץ ליישם שכבת Route Guard ייעודית בהמשך | | React escapes | XSS Prevention  
 Input Validation | | by default | מתבצע בשרת; מומלץ להרחיב ולחזק גם ברמת טפסי לקוח |

env

## משתנה סביבה מוכח בשימוש:

#

# אין שימוש חוכא ב-VITEAPIBASE\_URL

---



# כִּי הֵ-`baseURL` מוגדר ישירות בתוך `os.ts`

---

# ומצביע לשרת:Render

## Scripts 2.11

```

bash
npm run dev      # Development server
npm run build    # Production build command
npm run preview  # Preview build
npm run lint     # ESLint

```

## 2.12 הצעות להמשך (Client Roadmap)

| נושא | מצב קיים | שיפור מוצע | |-----|-----|-----|  
 | router | השלמת שכבות `pages/services/types/features` | תחת חוזים קבועים | | Route |  
 | Protection | הרשאות נאכפות בעיקר בשרת | הוספת `ProtectedRoute` ו- `AdminRoute` בצד לקוח |  
 | ניהול `baseURL` | API Base URL קשיח בקובץ `axios` | מעבר ל- `VITEAPIBASE_URL` לפי סביבות | | טיפול  
 | שגיאות אחיד | Toasts קיימים בחלק מהזרימות | `Error boundary + policy` אחיד לשגיאות HTTP | | בדיקות  
 | לקוח | תשתית בסיסית קיימת | הוספת `unit/component tests` ל- `flows` מרכזיים | ---

# חלק ד' - ניהול פרויקט

## 1.3.12 משאבים נדרשים

### 1.3.12.1 חלוקת שעות

-----|-----|-----|-----| | משימה | שעות | סטודנט 1 (Server) | סטודנט 2 (Client) |  
--| | תכנון וארכיטקטורה | 30 | 20 | 10 | | 100 | - | 100 | Frontend | - | 120 | 120 | Backend API | | בדיקות |  
| 30 | 20 | 10 | | 3 | 7 | 10 | Deployment | | תיעוד | 10 | 6 | 4 | | סה"כ | 300 | 173 | 127 |

### 1.3.12.2 ציוד נדרש

- מחשב עם 8GB+ RAM - חיבור אינטרנט (ל-APIs חיצוניים) - חשבון GitHub - חשבון MongoDB Atlas  
(חינם) - חשבון Stripe (test mode) - חשבון Render.com (חינם)

### 1.3.12.3 תוכנות נדרשות

| תוכנה | שימוש | | | |-----|-----| |  
VS Code | IDE | | Node.js 22+ | Runtime | | Git | Version control | | |  
| Postman | API testing | | Chrome DevTools | Debugging |

### 1.3.12.4 ידע חדש שנלמד

| נושא | מקור למידה | | | |-----|-----| |  
Express.js | expressjs.com | | MongoDB + Mongoose | | |  
mongodb.com/docs | | JWT Authentication | jwt.io | | Stripe Payments | stripe.com/docs | | TypeScript |  
| typescriptlang.org | | React + Redux Toolkit | redux-toolkit.js.org | | Material UI | mui.com |

### 1.3.12.5 ספרות ומקורות

Express.js Documentation - MongoDB University (free courses) - Stripe Documentation - OWASP -  
Top 10 (security) - "Clean Code" - Robert C. Martin - Node.js Best Practices -  
--- goldbergonyoni/nodebestpractices

## 1.3.13 תכנית עבודה

### Sprint Plan

## 1.3.14 תכנון בדיקות

### 1.3.14.1 בדיקות תהליכיות (Full Flow)

# | תרחיש | צעדים | תוצאה צפויה | |-----|-----|-----|---| | POST /register → | הרשמה | 1 | |-----|-----|-----|---| | POST /login → GET /verify | JWT valid, user data | | | | | Login | User created, token issued | | 2  
Add to cart → Checkout → Pay | Order confirmed, stock reduced | | 4 | Brute force | 5 | | 3  
wrong passwords | Account locked 15 min | | 5 | Logout | POST /logout → Use old token | 401  
| Unauthorized | | 6 | Webhook | Stripe event → Order update | Payment processed once

### 1.3.14.2 בדיקות יחידה (Unit Tests)

מודול | בדיקות מתוכננות / קיימות | |-----|-----| | **Auth Service** | Login, Register, | |-----|-----| | Lockout, tokenVersion, Google auth | | **Order Service** | Create, Cancel, Track order, Stock validation | |  
**Payment Service** | Webhook verify, Idempotency, Payment status | | **Product Service** | List, Read,  
| Filtering | | **Health / Integration** | Health checks, integration flow, performance scenarios

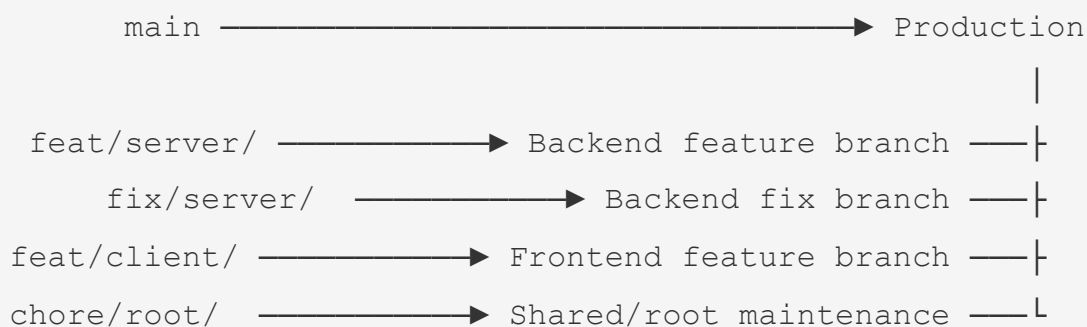
## Test Results

```
:Test files currently present in the repository
    auth.test.ts -
    auth.test.google.ts -
    products.test.ts -
    health.test.ts -
    order.test.ts -
    payment-webhook.test.ts -
    integration.test.ts -
    performance.test.ts -
```

.Jest is configured to run compiled tests from dist/**tests**  
fig.js, but no exact percentage is asserted here without a fresh test run

## 1.3.15 בקרת גרסאות

### Git Workflow



### Commit Convention

```

feat: Add account lockout after 5 failed logins
fix: Correct race condition in order creation
docs: Update API documentation
test: Add tests for payment webhook
refactor: Extract validation to separate module
  
```

### Version Numbering

**Semantic Versioning (SemVer)** - Major (X.0.0) – Breaking changes - Minor (0.X.0) – New features -  
 --- Patch (0.0.X) – Bug fixes **Current Version:** 1.1.0

## 1.3.16 תכנית הפקה סופית לספר (מבוסס קוד מקור)

מטרת פרק זה היא להגדיר תהליך עבודה אחד, מסודר ומדויק, עד לגרסת PDF סופית.

### 1.3.16.1 עקרונות מנחה

- מקור האמת הוא קוד המקור בריפו. - Swagger משמש להצלבה בלבד, לא כתחליף לקריאת קוד. - כל טענה בספר חייבת להיות מקוּבָּע בקטע קוד, route, model או service קיים.

| שלב | תוצר | קריטריון סיום | |----|-----| | 1 | נעילת מבנה הספר (מבוא/גוף/סיכום) | יש  
 כותרות קבועות לכל החלקים | | 2 | API Spec מלא | לכל endpoint יש method, auth, קלט, פלט ושגיאות |  
 3 | דיאגרמת DB | כל הישויות והקשרים מופיעים לפי models בפועל | | 4 | דיאגרמות זרימה בשרת, Login  
 Checkout, Webhook מתוארים שלב-שלב | | 5 | הטמעת קטעי קוד | לכל פרק טכני יש קוד אמיתי  
 מהמערכת + הסבר קצר | | 6 | פרק חלוקת עבודה | מוגדר בבירור מי אחראי על כל שכבה | | 7 | סיכום  
 רפלקטיבי | מה למדנו, אתגרים, הצעות להמשך | | 8 | QA לפני PDF | בדיקת עקביות בין הספר, הקוד והמצב  
 בפועל |

### 1.3.16.3 כללי איכות כתיבה (לפי הנחיית מנחה)

- מבוא: כתיבה יזמית, מדגישה ערך עסקי ומשמעות. - גוף מרכזי: כתיבה טכנית בסגנון CTO, מזויקת ועם ראיות מהקוד. - סיכום: כתיבה סטודנטיאלית רפלקטיבית, עם תובנות והמלצות.

### 1.3.16.4 כללי הטמעת קוד במסמך

- לא נדרשים צילומי מסך של קוד אם מציגים בלוקי קוד קריאים במסמך. - כל קטע קוד יוצג עם כותרת קצרה: "מה הבעיה" ו-"למה זה חשוב". - לכל קטע קוד תתווסף פסקת הסבר קצרה (3-5 שורות), ללא העתקת קובץ שלם. - אורך מומלץ לקטע: 15-40 שורות, ממוקד לפונקציה או מנגנון אחד. ---

### 1.3.17 קטעי קוד מומלצים להטמעה

הסעיף הבא מבוסס על קבצים שקיימים בפועל בריפון, ומטרתו להבטיח שהספר יישען על מימוש אמיתי.

## Backend - Models 1.3.17.1

| קובץ | מה להציג | למה זה חשוב | |-----|-----|-----|  
 server/src/models/user.model.ts | tokenVersion, failedLoginAttempts,  
 | lockedUntil, comparePassword | מראה אבטחת התחברות ו-logout מידי |  
 server/src/models/product.model.ts | sku, stock, isActive | ואינדקסים | תומך במלאי,  
 server/src/models/order.model.ts | status, paymentStatus, | סיונו וביצועים |  
 | fulfilled, trackingHistory | מציג מחזור חיים של הזמנה |  
 server/src/models/payment.model.ts | מזהי ספק תשלום וסטטוסים | תיעוד שכבת תשלומים |  
 server/src/models/webhook-event.model.ts | eventId | ייחודי | הבסיס ל-idempotency  
 | webhooks-כ

## Backend - Services 1.3.17.2

||-----|-----|-----| למה זה חשוב |  
 login | server/src/services/auth.service.ts | עם lockout + logout () עם העלאת  
 tokenVersion | מנגנון אבטחה מרכזי | server/src/services/order.service.ts ||  
 createOrder () עם בדיקות מלאי ויצירת pending\_payment | מפריד בין יצירת הזמנה לאישור  
 server/src/services/payment.service.ts | handleWebhook() ||  
 (signature/idempotency/amount check) | אמינות תשלום ומניעת חיוב כפול ||  
 server/src/services/payment.service.ts | בלוק ה-fulfillment עם transaction | מניעת  
 overselling ושמירת עקביות || addToCart | server/src/services/cart.service.ts () או  
 clearCart () | לוגיקת עגלה ומעבר להזמנה |

### Backend - Controllers + Routes 1.3.17.3

||-----|-----|-----| למה זה חשוב |  
 server/src/routes/auth.routes.ts | חלוקה ל-public/protected | מיפוי הרשאות ברור ||  
 server/src/routes/order.routes.ts | idempotencyMiddleware("order") + track  
 server/src/routes/payment.routes.ts || שילוב אמינות עם גישת משתמש ||  
 webhooks ציבורי מול create-intent מאומת | הפרדת תעבורה חיצונית/פנימית ||  
 server/src/controllers/payment.controller.ts | כניסה לשכבת  
 webhooks/createIntent/getStatus | מיפוי HTTP ל-service |  
 server/src/controllers/auth.controller.ts | login/logout/verify | נקודות כניסה לאימות  
 והרשאות |

### Frontend 1.3.17.4 - קטעים רלוונטיים

||-----|-----|-----| למה זה חשוב |  
 client/src/api/axios.ts || request interceptor עם Bearer token | מראה חיבור מאובטח ל-client/src/App.tsx || API | מבנה  
 routes + טעינת user/cart ב-useEffect | מראה orchestration של אפליקציה ||  
 client/src/components/GoogleConnectBtn.tsx | טעינת Google script ו-login callback |  
 client/src/components/AddToCartButton.tsx || דוגמה לשירות חיצוני בצד לקוח ||  
 לעגלה וריענון מצב | flow מוצר → עגלה ||  
 client/src/components/admin/ProductsTable.tsx | ניהול מוצרים בצד admin | דוגמה  
 ליכולות ניהול |

### 1.3.17.5 מבנה מומלץ להצגת כל קטע קוד

לכל קטע קוד במסמך: 1. כותרת קצרה (לדוגמה: "Webhook Idempotency"). 2. פסקת הקשר קצרה (מה הפונקציה עושה). 3. בלוק קוד אמיתי מהקובץ (ללא שכתוב מלאכותי). 4. הסבר קצר על תרומה



## 1.3.17.6 קטעי קוד בפועל - Backend

#### קטע 1: User Model - Lockout + Token Version מקור:

server/src/models/user.model.ts

```
typescript
Account lockout //
} :failedLoginAttempts
, type: Number
, default: 0
, min: 0
, {

} :lockedUntil
, type: Date
, default: null
, index: true
, {

Token version: incrementing invalidates all existing tokens instantly //
} :tokenVersion
, type: Number
, default: 0
, {
```

הסבר קצר: המודל שומר גם נתוני נעילה נגד brute force וגם tokenVersion שמאפשר invalidate מיידית לכל הטוקנים אחרי logout. #### קטע 2: AuthService.login - נעילת חשבון אחרי כשלונות מקור:

server/src/services/auth.service.ts

```
typescript
Check if account is locked  //
} if (user.lockedUntil && new Date() < user.lockedUntil)
) const remainingMinutes = Math.ceil
, 60000 / ((user.lockedUntil.getTime() - Date.now)
; (
) throw new ApiError
```

```

ailed login attempts. Please try again in ${remainingMinutes} minutes
                                ,undefined
                                ,"ACCOUNT_LOCKED"
                                ; (
                                {

                                Verify password //
;const isPasswordValid = await user.comparePassword(credentials.password)
                                } if (!isPasswordValid)
;user.failedLoginAttempts = (user.failedLoginAttempts || 0) + 1

                                } if (user.failedLoginAttempts >= 5)
;user.lockedUntil = new Date(Date.now() + 15 60 1000)
                                ;()await user.save
                                )throw new ApiError
                                ,423
to too many failed login attempts. Please try again in 15 minutes"
                                ,undefined
                                ,"ACCOUNT_LOCKED"
                                ; (
                                {

                                ;()await user.save
;throw new UnauthorizedError("Invalid email or password.")
                                {

```

הסבר קצר: בתרחישי סיסמה שגויה המערכת צוברת ניסיונות כושלים, נועלת ל-15 דקות אחרי 5 כשלים, ומחזירה 423 במקום להמשיך לאפשר ניסיונות בלתי מוגבלים. #### קטע 3: Logout + Verify Token

Version מקור: server/src/services/auth.service.ts

```

typescript
} static async logout(userId: string)
;const user = await UserModel.findById(userId)
} if (!user)
;throw new Error("User not found")
{

```

```

;user.tokenVersion = (user.tokenVersion || 0) + 1
;()await user.save

;return { message: "Logged out successfully - all sessions revoked" }
{

} static async verifyToken(token: string)
{ const decoded = jwt.verify(token, JWT_SECRET) as
;userId: string
;tokenVersion?: number
;{
;const user = await UserModel.findById(decoded.userId)

} if (!user || !user.isActive)
;throw new Error("Invalid token")
{

) if
&& decoded.tokenVersion !== undefined
decoded.tokenVersion !== user.tokenVersion
} (
;throw new Error("Token revoked")
{

;return this.sanitizeUser(user)
{

```

הסבר קצר: ה-logout לא שומר blacklist אלא מעלה גרסה (tokenVersion), וכל טוקן ישן נכשל מיד ב-

verifyToken. #### קטע 4: יצירת Order מנותקת מאישור תשלום מקור:

server/src/services/order.service.ts

```

typescript

Verify stock (but DON'T reduce it yet!)  //
;const products = await ProductModel.find({ _id: { $in: productIds } })

} for (const item of cart.items)

```

```

;const product = productMap.get(productId)

    } if (!product || !product.isActive)
;throw new ApiError(400, Product `${item.product} is not available`)

    {

        } if (product.stock < item.quantity)
;throw new ApiError(400, Insufficient stock for `${product.name}`)

    {

        })orderItems.push
        ,product: product._id
        ,name: product.name
        ,price: item.lockedPrice || product.price
        ,quantity: item.quantity
        ,image: product.image
        ;({
        {

    })const order = await OrderModel.create
        ,orderNumber
        ,user: userId
        ,items: orderItems
        ,totalAmount
        ,"status: "pending_payment
        ,"paymentStatus: "pending
        ,"paymentMethod: orderData.paymentMethod || "stripe
        ,shippingAddress: orderData.shippingAddress
        ;({

```

הסבר קצר: ההזמנה נוצרת בסטטוס `pending_payment` לאחר בדיקות מלאי, אך הפחתת מלאי

מתבצעת רק אחרי אישור תשלום מה-`webhook`. #### קטע 5: Webhook Controller - Duplicate Handling

server/src/controllers/payment.controller.ts מקור: + Retry Queue

```

typescript
    } try

;const result = await PaymentService.handleWebhook(req)

```

```

;res.status(200).json({ received: true, message: "Duplicate event ignored" });
    } catch (error: any) {
        // Return 200 for duplicate events (already processed) //
        if (error.message?.includes("already processed"))
            return res
                .status(200)
                .json({ received: true, message: "Duplicate event ignored" });

        // Save failed webhook for retry //
        await FailedWebhookModel.create(
            {
                eventId,
                eventType,
                provider,
                payload: rawBody,
                error: error.message,
                retryCount: 0,
                maxRetries: 5,
                nextRetryAt,
                status: "pending"
            }
        );

        ;res.status(400).json({ received: false, error: error.message });
    }
}

```

הסבר קצר: אם האירוע כבר עובד בעבר המערכת מחזירה 200 כדי למנוע retries מיותרים; כשל אמיתי נשמר לטבלת FailedWebhook כדי לאפשר retry מתוזמן. #### קטע 6: PaymentService - Amount Verification + Atomic Fulfillment מקור: server/src/services/payment.service.ts

```

typescript
;const expectedAmountInCents = Math.round(order.totalAmount * 100)
;const receivedAmountInCents = result.amount || 0

} if (receivedAmountInCents !== expectedAmountInCents)
    ;"payment.status = "failed"
    ;()await payment.save

```

```

; "order.paymentStatus = "failed"
; ()await order.save

)throw new Error

expectedAmountInCents} cents, received ${receivedAmountInCents} cents

; (
{

; ()const session = await mongoose.default.startSession
; ()session.startTransaction

} for (const item of order.items)
)const product = await ProductModel.findByIdAndUpdate
, item.product
,{ inc: { stock: -item.quantity }$ }
,{ new: true, session }
; (

} if (!product || product.stock < 0)
;throw new Error(Insufficient stock for ${item.name})
{
{

;await CartService.clearCart(order.user.toString())
;order.fulfilled = true
; ()order.fulfilledAt = new Date
;await order.save({ session })

; ()await session.commitTransaction

```

הסבר קצר: תחילה נעשית בדיקת סכום מחמירה מול ההזמנה בבסיס הנתונים; לאחר מכן הפחתת מלאי וסגירת הזמנה מתבצעות בתוך transaction כדי למנוע מצב ביניים לא עקבי. #### קטע 7: הפרדת Public ו-

Protected במסלולי Auth מקור: `server/src/routes/auth.routes.ts`

```

typescript
Public routes //
;router.post("/register", authRateLimiter, AuthController.register)

```

```

;router.post("/login", authRateLimiter, AuthController.login)
;router.post("/forgot-password", authRateLimiter, AuthController.forgotPassword)
;router.post("/reset-password/:token", authRateLimiter, AuthController.resetPassword)
;router.post("/refresh", authRateLimiter, AuthController.refreshToken)
;router.post("/google", authRateLimiter, AuthController.googleLogin)

// Protected routes
;router.get("/verify", authenticate, AuthController.verify)
;router.get("/profile", authenticate, AuthController.getProfile)
;router.put("/profile", authenticate, AuthController.updateProfile)
;router.post("/change-password", authenticate, AuthController.changePassword)
;router.post("/logout", authenticate, AuthController.logout)

```

הסבר קצר: הקובץ מפריד באופן מפורש בין נקודות פתוחות לנקודות שמחייבות token תקף, ומחיל

על נתיבי auth רגישים. --- authRateLimiter

# חלק ה' - סיכום

## סיכום - מה למדנו

### מסקנות טכניות

במהלך הפרויקט נרכש ניסיון מעשי בתכנון ובמימוש מערכת תוכנה רב-שכבתית: **צד שרת:** -  ארכיטקטורת Layered Architecture עם הפרדה ברורה בין שכבות -  אבטחה מרובת שכבות: JWT, idempotency -  Stripe כולל webhooks ו- - bcrypt, rate limiting, tokenVersion אינטגרציה עם Stripe כולל webhooks ו- MongoDB transactions למניעת race conditions -  כתיבת בדיקות אוטומטיות עם Jest **צד לקוח:** -  React 19 עם Redux Toolkit + useState -  TypeScript לניהול CSS + Material UI  - state לעיצוב responsive -  עבודה עם טוקנים, axios interceptor והפרדת אזורי משתמש/אדמין ברמת ה-routing המתוכנן

### מסקנות תהליכיות

מעבר להיבט הטכנולוגי, הפרויקט חידד גם היבטים תהליכיים ומתודולוגיים: נושא | מה למדנו | |-----|-----|  
 |----| | **עבודת צוות** | חלוקת עבודה, Git workflow, code review | | **תכנון** | לחשוב קדימה, לצפות בעיות, לתכנן פתרונות | | **תיעוד** | הקוד הוא רק חצי מהעבודה - תיעוד חשוב לא פחות | | **אבטחה** | לחשוב כמו תוקף, לא לסמוך על קלט מהמשתמש | | **Production** | ההבדל בין "עובד על המחשב שלי" ל-production |

### אתגרים עיקריים

| אתגר | איך התמודדנו | |-----|-----| | Race conditions | למדנו על | MongoDB transactions | Token security | חקרנו ומצאנו את שיטת Webhook reliability | | tokenVersion | הבנו idempotency | TypeScript errors | | patterns | זיהינו פערים בין מבנה הקליינט בפועל לבין הייבואים והטיפוסים שמוגדרים בו |

### המלצות להמשך ולפרויקטים עתידיים

1. **תכננו את הארכיטקטורה לפני שמתחילים לקודד** - זה חוסך שעות של 2 refactoring. **כתבו בדיקות מההתחלה** - לא בסוף כשכבר אין זמן 3. **תעדו תוך כדי** - לא לזכור בסוף מה עשיתם לפני 3 חודשים 4. **אל תזלזלו באבטחה** - זה לא "נוסף אחר כך" 5. **Deploy מוקדם** - תגלו בעיות production לפני שמאוחר מדי

### סיכום



**מה קט זה עושה** כמסגרת ליישום מעשי של עקרונות הנדסת תוכנה בסביבת פיתוח מציאותית יחסית, הכוללת חלוקה בין צד שרת לצד לקוח, אינטגרציה עם שירותים חיצוניים, תיעוד, בדיקות, וניהול גרסאות. נכון למצב הריפו הנוכחי, צד השרת מתועד וממומש באופן מלא ומגובש יותר, בעוד שבצד הלקוח קיימת תשתית פונקציונלית טובה, עם שכבות נוספות שעדיין נמצאות בתהליך השלמה כחלק מחלוקת העבודה בין המפתחים. למרות פער זה, הפרויקט מספק בסיס טכני ותיעודי משמעותי, ומדגים הבנה מעשית של תכנון מערכת, מימוש API, אבטחת מידע, תהליכי שלום, וניהול עבודה משותפת בסביבת Git. --- **תאריך עדכון**

**אחרון:** 10/05/2026 **מצב:** Final **גרסה:** 1.1.0